Pontifícia Universidade Católica do Rio Grande do Sul · Escola Politécnica
46504-04 Construção de Software · Turma 330 · 2026/2
Prof. Michael da Costa Mora

PulsePass

Plataforma Distribuída de Bilhetagem, Reserva e Acesso para Megaeventos

Pacote 2: Arquitetura e Infraestrutura de Desenvolvimento

Grupo 9
Gabriel Gerhardt de Marque
Mateus Granja de Andrade Madalosso Tieppo
Filipe Tesche Roman

Repositório: **https://github.com/Mateus-Tieppo/construcao-de-software** (pasta `pacote-2/`)

Porto Alegre, 4 de novembro de 2026

Sumário

1. Descrição da arquitetura no modelo C4
2. Descrição do Modelo de Dados Conceitual
3. Descrição da estrutura de mensageria do Kafka
4. Descrição esquemática da Pipeline de desenvolvimento
5. Descrição do Processo de GCS

Convenção de rótulos. [Enunciado] marca exigência do professor; **[Decisão do grupo, Pacote 1]** marca escolha já registrada no Pacote 1; **[Proposta para revisão do grupo]** marca escolha nova deste pacote, ainda não aprovada pelo grupo.

Os ADRs, os contratos de eventos de exemplo e a auditoria de verificação deste pacote estão no repositório, em `pacote-2/adr/`, `pacote-2/contratos/` e `pacote-2/06-verificacao-entrega.md`. Nenhum código de aplicação foi implementado neste pacote; tudo o que descreve execução é planejado.

1. Descrição da Arquitetura no Modelo C4

> Versão 1.0 · 04/11/2026 · Grupo 9 · Será atualizado nos Pacotes 3 e 4. Rótulos: **[Enunciado]** exigência do professor · **[Decisão do grupo, Pacote 1]** escolha já registrada · **[Proposta para revisão do grupo]** escolha nova deste pacote. As decisões novas estão detalhadas nos ADRs de `pacote-2/adr/`. Este documento descreve a arquitetura **planejada**. Nenhum serviço foi implementado até esta entrega.

1.1 Objetivo e leitura dos diagramas

O modelo C4 descreve o PulsePass em três níveis de zoom **[Enunciado]**:

Nível	Pergunta que responde	Seção
C1 · Contexto	Quem usa o sistema e com quais sistemas externos ele conversa?	1.2
C2 · Contêineres	Quais processos implantáveis e bancos existem e como se comunicam?	1.3
C3 · Componentes	Como cada serviço de backend é organizado por dentro?	1.4

Os diagramas são escritos em Mermaid (diagramas como código, versionados com o resto do projeto). Usamos sempre a mesma legenda:

Forma / cor	Significa
Azul escuro	Pessoa (perfil RBAC)
Azul	Sistema PulsePass ou contêiner desenvolvido pelo grupo
Cinza	Sistema ou produto externo, não desenvolvido pelo grupo
Cilindro	Armazenamento de dados
Seta com rótulo	Finalidade da relação e protocolo

Um **contêiner C4** é qualquer coisa que roda ou guarda dados separadamente (uma aplicação, um banco, um broker). Ele não é necessariamente um contêiner Docker, embora no PulsePass quase todos virem um serviço do `docker-compose.yml`.

1.2 Nível 1: Contexto

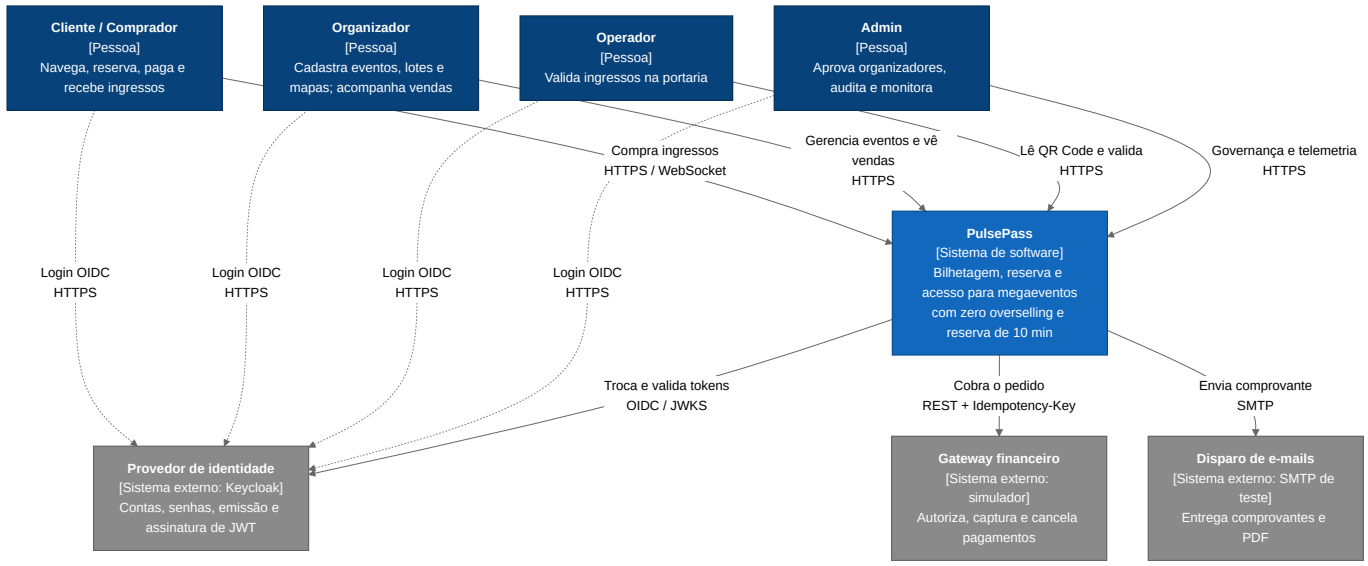


Figura 1. C1: contexto do PulsePass. Escopo: o sistema inteiro e seus vizinhos.

Elemento	Tipo	Pertence ao PulsePass?	Observação
Cliente, Organizador, Operador, Admin	Pessoas	Não	Os quatro papéis RBAC **[Enunciado]**
PulsePass	Sistema	Sim	Desenvolvido pelo grupo
Provedor de identidade	Externo	Não	Keycloak, produto de terceiros executado localmente **[Decisão do grupo, Pacote 1 (R04)]**
Gateway financeiro	Externo	Não (simulado)	Representa um gateway bancário. O grupo escreve um simulador, mas ele fica fora da fronteira lógica do produto **[Decisão do grupo, Pacote 1 (P06)]**
Disparo de e-mails	Externo	Não	Em desenvolvimento, Mailpit; em produção seria um provedor SMTP

1.3 Nível 2: Contêineres

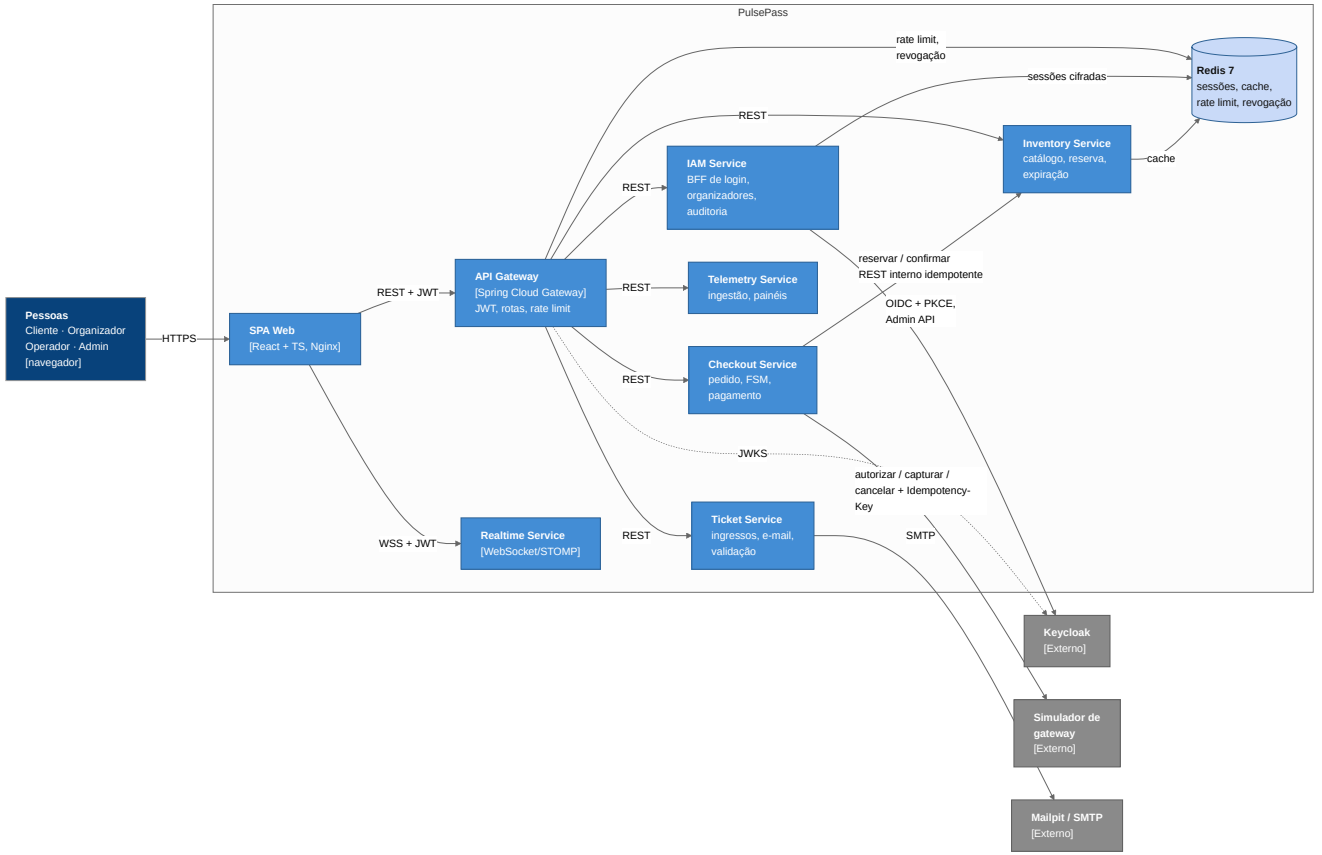


Figura 2a. C2: contêineres e chamadas síncronas. Todos os contêineres de backend validam o JWT pela chave pública (JWKS) do Keycloak; só a seta do Gateway aparece para não poluir o diagrama.

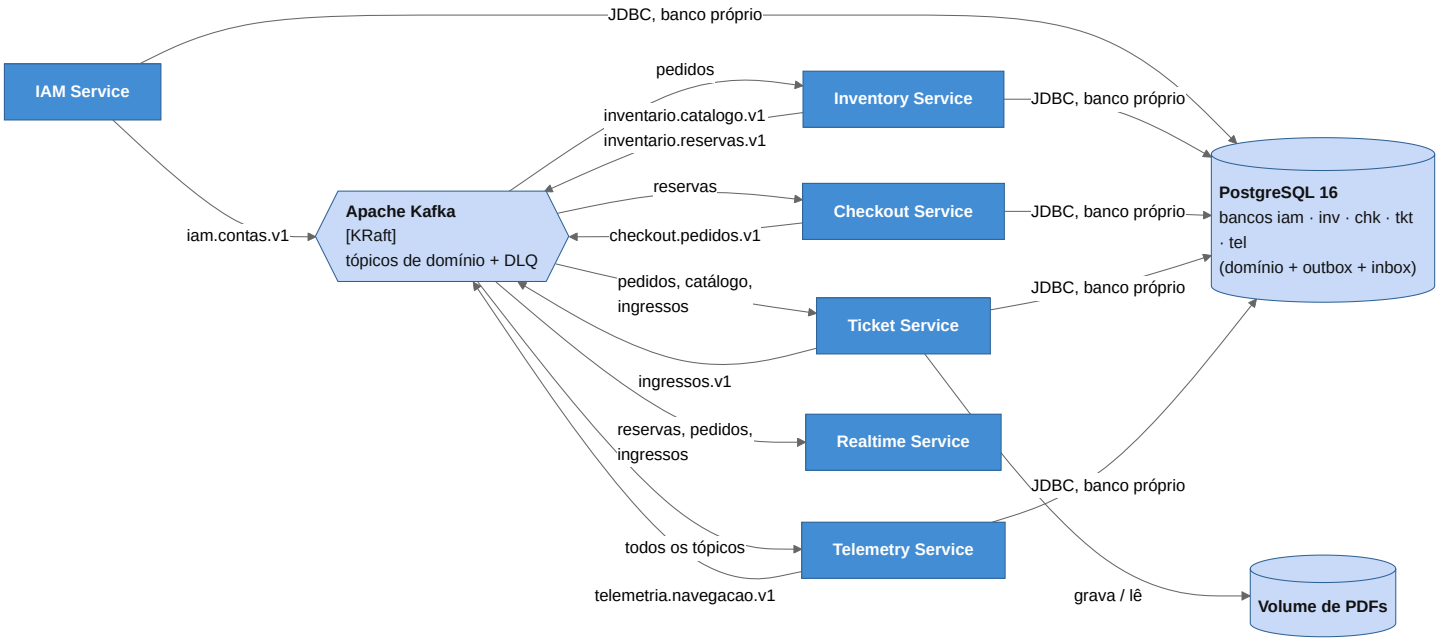


Figura 2b. C2: dados e mensageria. Os serviços que publicam (à esquerda) usam a Outbox no próprio banco; os que consomem usam a Inbox (documento 3). Observabilidade e Kafka UI estão na Figura 3.

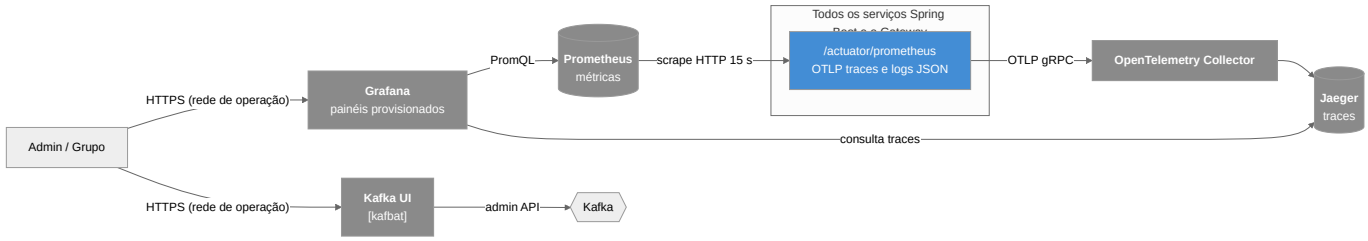


Figura 3. C2 (complemento): contêineres de operação e observabilidade.

1.3.1 Responsabilidade de cada contêiner

Contêiner	Módulo do enunciado	Tecnologia	Responsabilidade	Dados que possui	Rótulo
SPA Web	Todos (front-end)	React + TS (Vite), servida por Nginx	Telas dos 4 perfis (Pacote 1, item 5); guarda só o access token em memória	Nenhum persistente	[Decisão do grupo, Pacote 1 (P02)]
API Gateway	Transversal	Spring Cloud Gateway	Ponto único de entrada HTTP; valida JWT; consulta lista de revogação; *rate limiting* por usuário/IP (RNF11); propaga `traceparent`	Contadores no Redis	[Decisão do grupo, Pacote 1]
IAM Service	1. Identidade e Governança	Spring Boot 3, Java 21	BFF de autenticação (login, refresh, logout), solicitação e aprovação de organizadores, gestão de papéis via Admin API do Keycloak, trilha de auditoria	Banco `iam`; sessões no Redis	[Proposta para revisão do grupo] (ADR-004)
Inventory Service	2. Inventário e Concorrência	Spring Boot 3	Eventos, setores, lotes, assentos, catálogo de alta leitura, reserva atômica, confirmação, devolução e expiração	Banco `inv`; cache no Redis	[Enunciado] + ADR-002
Checkout Service	3. Transações e Checkout	Spring Boot 3, Resilience4j	Pedido e FSM, Idempotency-Key, integração com simulador, Outbox, reconciliação	Banco `chk`	[Enunciado] + ADR-003
Ticket Service	4. Mensageria e Despacho	Spring Boot 3, OpenPDF, ZXing	Consome pagamentos, emite ingressos (PDF + QR assinado), envia e-mail, valida check-in	Banco `tkt`; volume de PDFs	[Enunciado]
Realtime Service	4. Mensageria e Despacho	Spring Boot 3, STOMP	Mantém conexões WebSocket e empurra mudanças de assentos, pedidos e ingressos	Nenhum persistente	[Proposta para revisão do grupo] (ADR-001)
Telemetry Service	5. Telemetria e DevOps	Spring Boot 3, Micrometer	Recebe eventos de navegação da SPA, consome eventos de domínio, agrega funil e vendas, expõe métricas de negócio	Banco `tel`	[Enunciado]
PostgreSQL	Infra	PostgreSQL 16	Fonte da verdade de cada serviço, Outbox e Inbox	5 bancos + banco `keycloak`	[Decisão do grupo, Pacote 1 (P03)]
Redis	Infra	Redis 7 (AOF ligado)	Sessões de refresh (cifradas), cache, *rate limit*, lista de revogação. **Não decide posse de assento**	Dados reconstruíveis, exceto sessões	[Proposta para revisão do grupo] (ADR-002, ADR-004)
Kafka	Infra	Apache Kafka 3.8 (KRaft)	Streaming de eventos de domínio	Tópicos (documento 3)	[Enunciado]
Kafka UI	Infra	kafbat/kafka-ui	Inspeção de tópicos, grupos, lag e DLQ	Nenhum	[Decisão do grupo, Pacote 1 (P04)]
Simulador de gateway	Externo	Spring Boot (código do grupo)	Autorizar, capturar, cancelar; modos de falha e atraso configuráveis	Banco em memória	[Decisão do grupo, Pacote 1 (P06)]

1.3.2 Protocolos e fronteiras de confiança

Fronteira	De → Para	Protocolo	Autenticação	Observação
Internet → borda	Navegador → SPA, Gateway, Realtime	HTTPS / WSS (TLS no Nginx de borda)	JWT Bearer; cookie de sessão `HttpOnly` só no `/auth/*`	Única fronteira exposta (RNF23)
Borda → serviços	Gateway → serviços	HTTP na rede Docker interna `backend`	JWT repassado; cada serviço revalida assinatura, `exp`, `aud` e papel (RNF18)	Serviços não publicam portas para fora no Compose
Serviço → serviço	Checkout → Inventory	HTTP interno	Token de *client credentials* do Keycloak (conta de serviço `checkout`, escopo `inventario.reservas`)	Endpoints `/internal/*` não passam pelo Gateway
Serviço → externo	Checkout → simulador; Ticket → SMTP	HTTP / SMTP	Chave de API do simulador via variável de ambiente	Nunca enviar dados de cartão reais (são simulados)
Serviço → infraestrutura	Serviços → PostgreSQL, Redis, Kafka	JDBC, RESP, Kafka	Usuário e senha por serviço (variáveis de ambiente, `.env` fora do Git); ACL Redis por prefixo	Usuário de banco só enxerga o próprio banco (ADR-001)
Operação	Grupo → Grafana, Kafka UI, Jaeger	HTTP em rede `ops`	Login do Grafana; Kafka UI só em ambiente local	Não expostos no ambiente de demonstração pública

1.3.3 Principais APIs expostas pelo Gateway

Serviço	Endpoints (resumo)	Papel exigido
IAM	`GET /auth/login`, `GET /auth/callback`, `POST /auth/refresh`, `POST /auth/logout`, `POST /organizadores/solicitacoes`, `PATCH /admin/organizadores/{id}`, `PUT /admin/usuarios/{id}/papeis`, `GET /admin/auditoria`	público / autenticado / admin
Inventory	`GET /eventos`, `GET /eventos/{id}`, `GET /eventos/{id}/setores/{s}/assentos`, `POST /organizador/eventos`, `POST /organizador/eventos/{id}/lotes`, `PUT .../mapa`, `POST .../publicacao`, `PUT /organizador/eventos/{id}/operadores`	público / organizador dono
Checkout	`POST /pedidos` (com `Idempotency-Key`), `GET /pedidos/{id}`, `POST /pedidos/{id}/pagamento` (com `Idempotency-Key`), `POST /pedidos/{id}/cancelamento`	cliente dono

Serviço	Endpoints (resumo)	Papel exigido
Ticket	`GET /ingressos`, `GET /ingressos/{id}/pdf`, `POST /validacoes`, `GET /operador/ingressos?codigo=`	cliente dono / operador vinculado
Telemetry	`POST /telemetria/eventos` (lote), `GET /organizador/eventos/{id}/painel`, `GET /admin/painel`	autenticado / organizador dono / admin

Os contratos detalhados serão gerados em OpenAPI 3 no build de cada serviço (RNF36) a partir do Pacote 3.

1.4 Nível 3: Componentes

Todos os serviços seguem **arquitetura hexagonal [Decisão do grupo, Pacote 1 (RNF34)]**: adaptadores de entrada (controllers, consumidores) → casos de uso (camada de aplicação) → domínio puro (sem Spring, JPA ou Kafka) → portas implementadas por adaptadores de saída. ArchUnit verifica essas dependências. Os diagramas abaixo mostram os componentes principais; a ordem de prioridade é Inventário, Checkout e Ingressos, que concentram os requisitos de concorrência e consistência.

1.4.1 Inventory Service

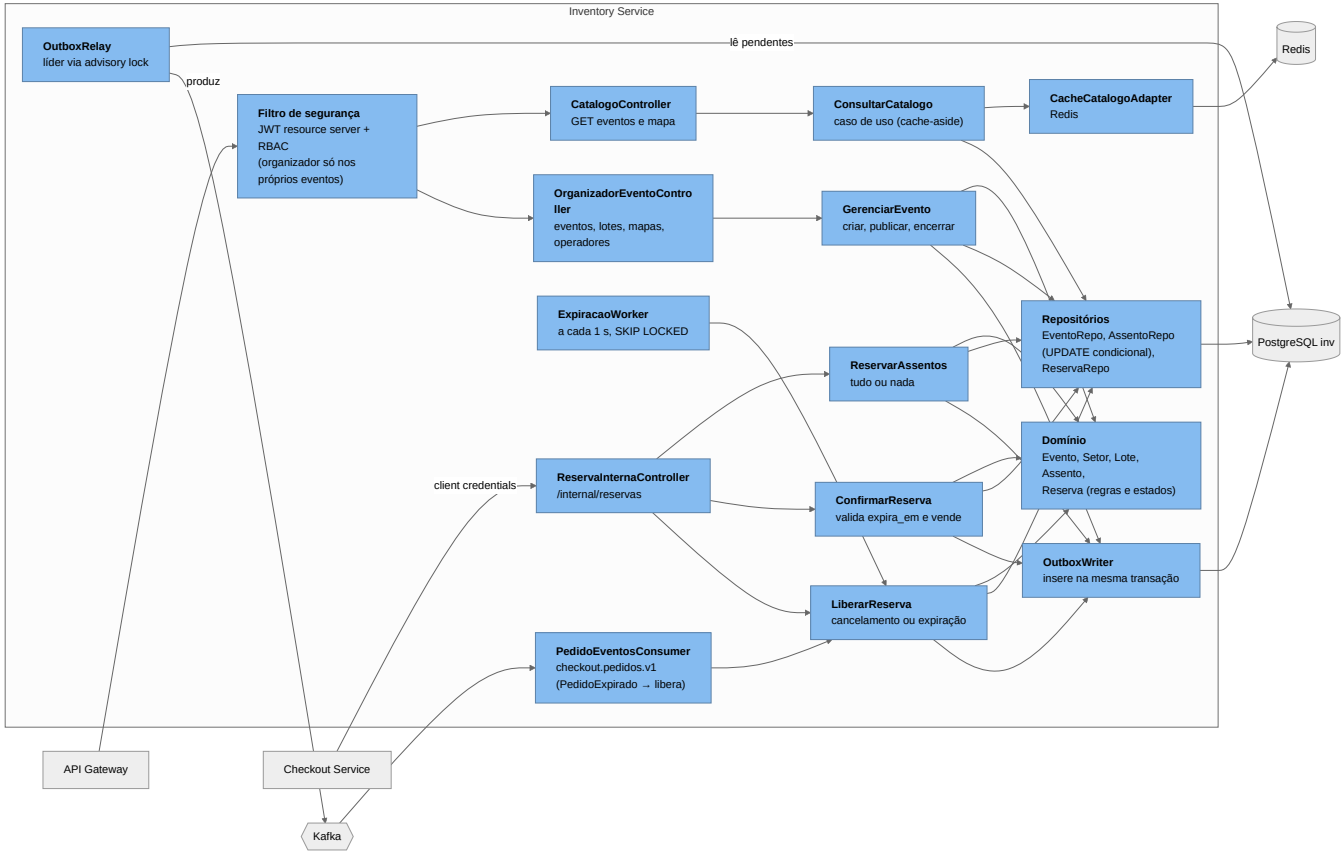


Figura 4. C3: componentes do Inventory Service.

Componente	Responsabilidade	Regra crítica
`ReservarAssentos`	Recebe `pedidoId`, `eventoId` e 1 a 6 assentos; abre transação; executa o `UPDATE` condicional em ordem de `id`; cria `Reserva` com `expira_em`; grava `ReservaCriada` na Outbox	Linhas afetadas = assentos pedidos, senão *rollback* (RNF01, RNF02, ADR-002). Idempotente por `pedidoId`
`ConfirmarReserva`	Chamado pelo Checkout após autorização do pagamento	Só confirma se `estado='ATIVA' AND now() < expira_em`; marca assentos `VENDIDO` (ADR-003)
`LiberarReserva`	Cancelamento pelo comprador, falha de pagamento definitiva ou expiração	Devolve todos os assentos da reserva na mesma transação; grava `ReservaExpirada` ou `ReservaLiberada`
`ExpiracaoWorker`	Varre reservas vencidas	Várias réplicas podem rodar: `FOR UPDATE SKIP LOCKED` impede duas limpezas da mesma reserva
`ConsultarCatalogo`	Leitura de alto volume	*Cache-aside* com TTL de 30 s para catálogo; mapa de assentos com TTL de 2 s e invalidação por evento; o banco continua sendo a verdade
`OutboxRelay`	Publica eventos pendentes	Um relay ativo por serviço, em ordem de `id` (ADR-005)

1.4.2 Checkout Service

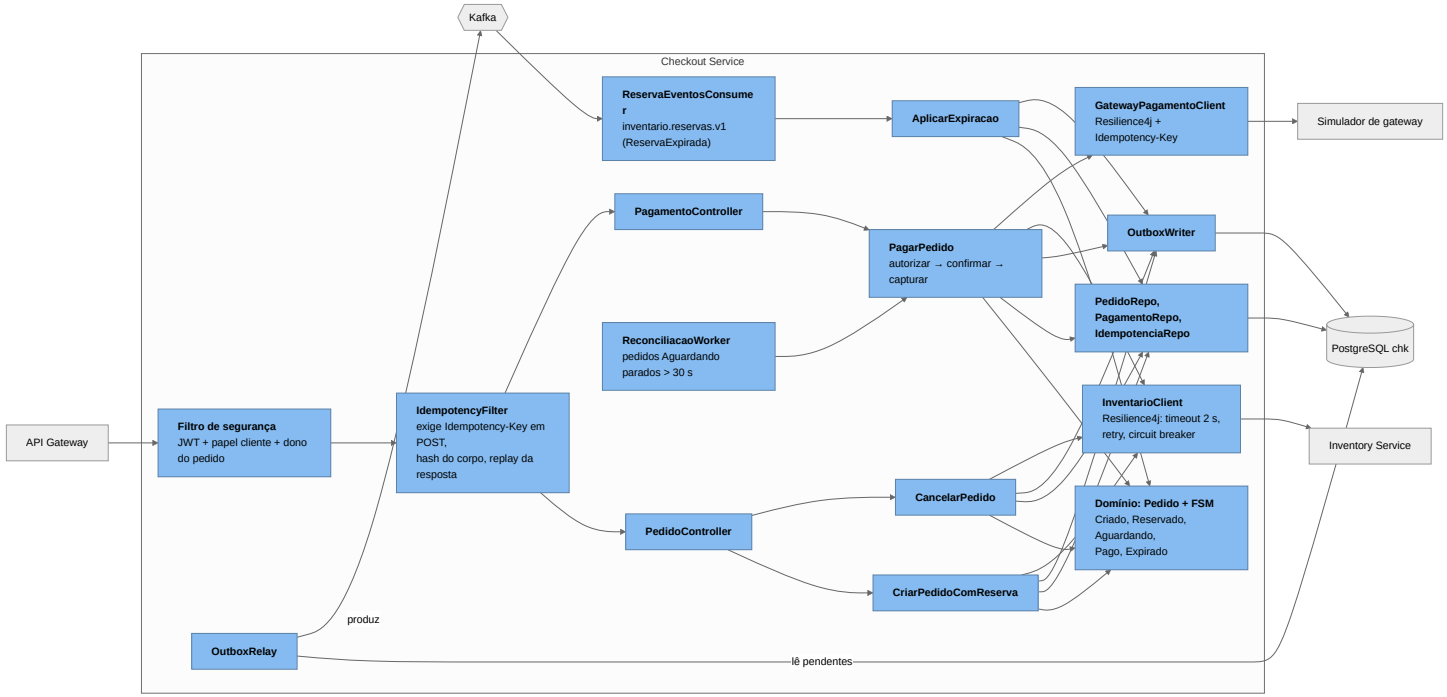


Figura 5. C3: componentes do Checkout Service.

Componente	Responsabilidade	Regra crítica
`IdempotencyFilter`	Antes do controller: valida presença da chave (400 se ausente, RNF05), grava registro `EM_ANDAMENTO` com hash SHA-256 do método, rota e corpo	Detalhes na seção 1.5.4
`CriarPedidoComReserva`	Cria pedido `Criado`, chama `ReservarAssentos` no Inventário, avança para `Reservado` com `expiraEm` copiado da reserva	Falha da reserva leva a `Expirado` com motivo `FALHA_RESERVA`
`PagarPedido`	`Reservado → Aguardando`, autoriza no simulador, confirma no Inventário, captura, `Aguardando → Pago` + `PagamentoConfirmado` na Outbox	Ordem e compensação do ADR-003
`AplicarExpiracao`	Consome `ReservaExpirada` e leva pedido não pago a `Expirado`	Transição guardada: evento sobre pedido `Pago` é ignorado e registrado (US21)
`ReconciliacaoWorker`	Resolve pedidos `Aguardando` após timeouts consultando o simulador e o Inventário	Nunca cria nova cobrança: usa sempre o mesmo `pagamentoId`
Domínio FSM	Puro Java, sem framework	Cobertura ≥ 90% de ramos, PIT ≥ 70% (RNF04, RNF32)

1.4.3 Ticket Service e Realtime Service

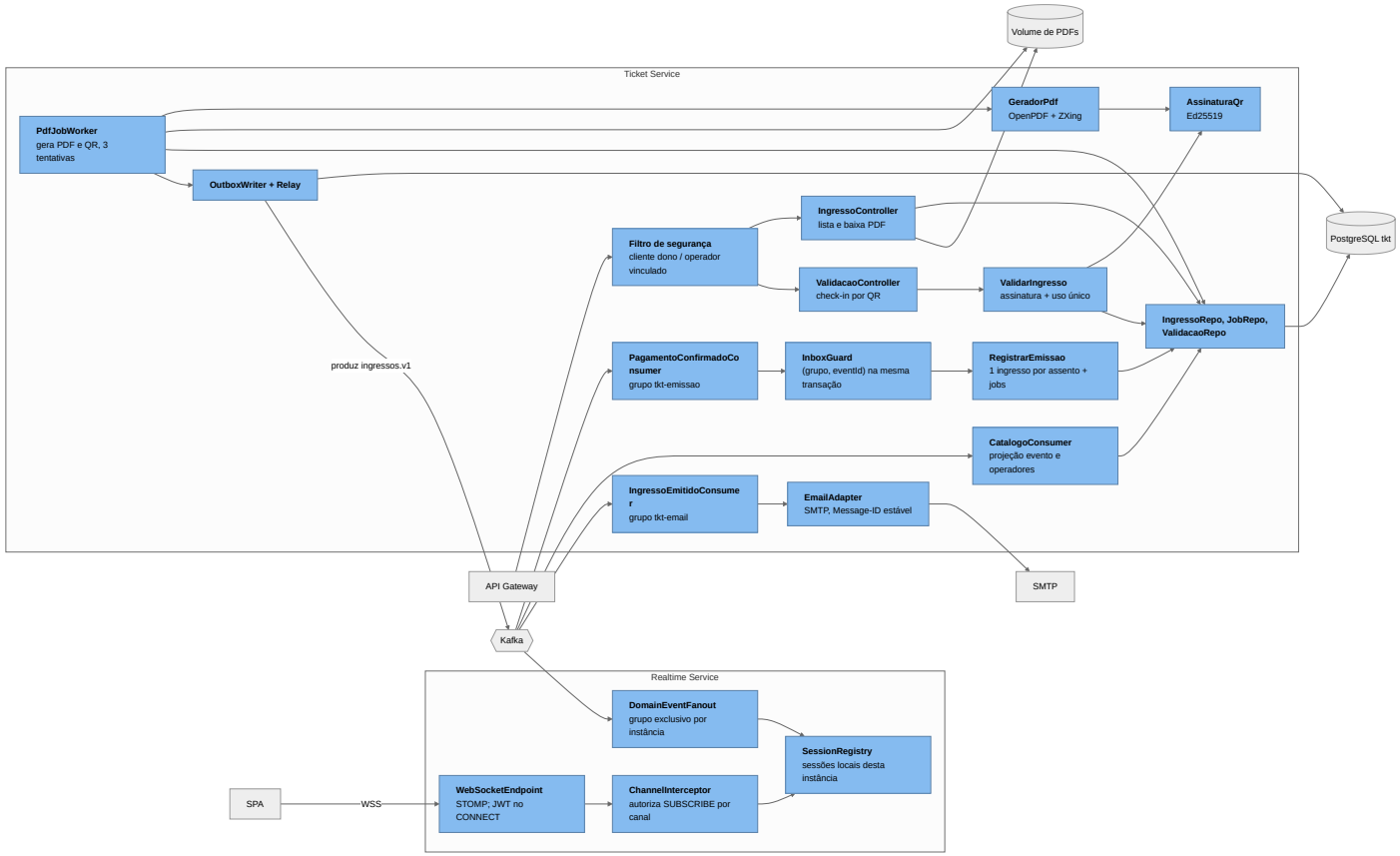


Figura 6. C3: componentes do Ticket Service e do Realtime Service.

Componente	Responsabilidade	Regra crítica
`PagamentoConfirmadoConsumer` + `InboxGuard` + `RegistrarEmissao`	Em uma transação: grava `(tkt-emissao, eventId)`, cria um ingresso `PENDENTE` por item de pedido e um job de PDF por ingresso	`UNIQUE (item_pedido_id)` garante um ingresso por assento mesmo com reprocessamento (RNF07)
`PdfJobWorker`	Gera o PDF (evento, setor, assento, titular, carimbo de tempo, QR assinado), grava em `ingressos/{ingressoId}.pdf`, marca `EMITIDO` e grava `IngressoEmitido` na Outbox	Mesmo `ingressoId` sobrescreve o mesmo arquivo; após 3 falhas o job vai a `FALHOU` e gera alerta (US25)
`ValidarIngresso`	Verifica assinatura, evento e vínculo do operador; `UPDATE ingresso SET estado='UTILIZADO' WHERE id=? AND estado='EMITIDO'`	Linha afetada = liberado; zero linhas = "já utilizado" ou inválido (RNF22, US29)
`IngressoEmitidoConsumer` + `EmailAdapter`	Envia e-mail com PDF anexado	Registra `NOTIFICACAO` como PENDENTE antes do SMTP e ENVIADA depois; limite de idempotência no documento 3, seção 3.7
`DomainEventFanout`	Cada instância do Realtime usa um consumer group próprio (`rt-{instanceId}`), lendo do fim do tópico	Todas as instâncias recebem todos os eventos e entregam só às suas sessões (seção 1.6)

1.4.4 IAM Service

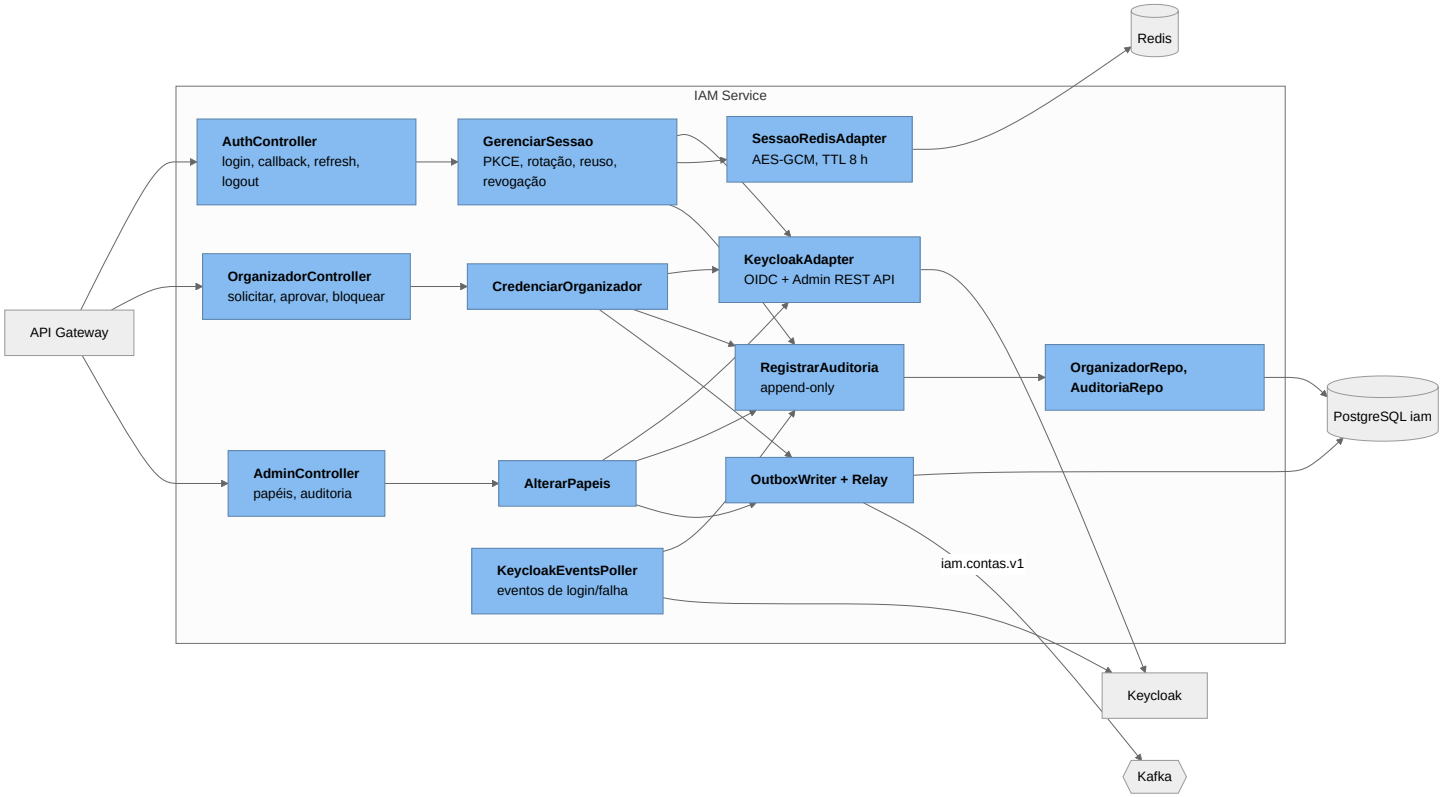


Figura 7. C3: componentes do IAM Service (ADR-004).

1.4.5 Telemetry Service

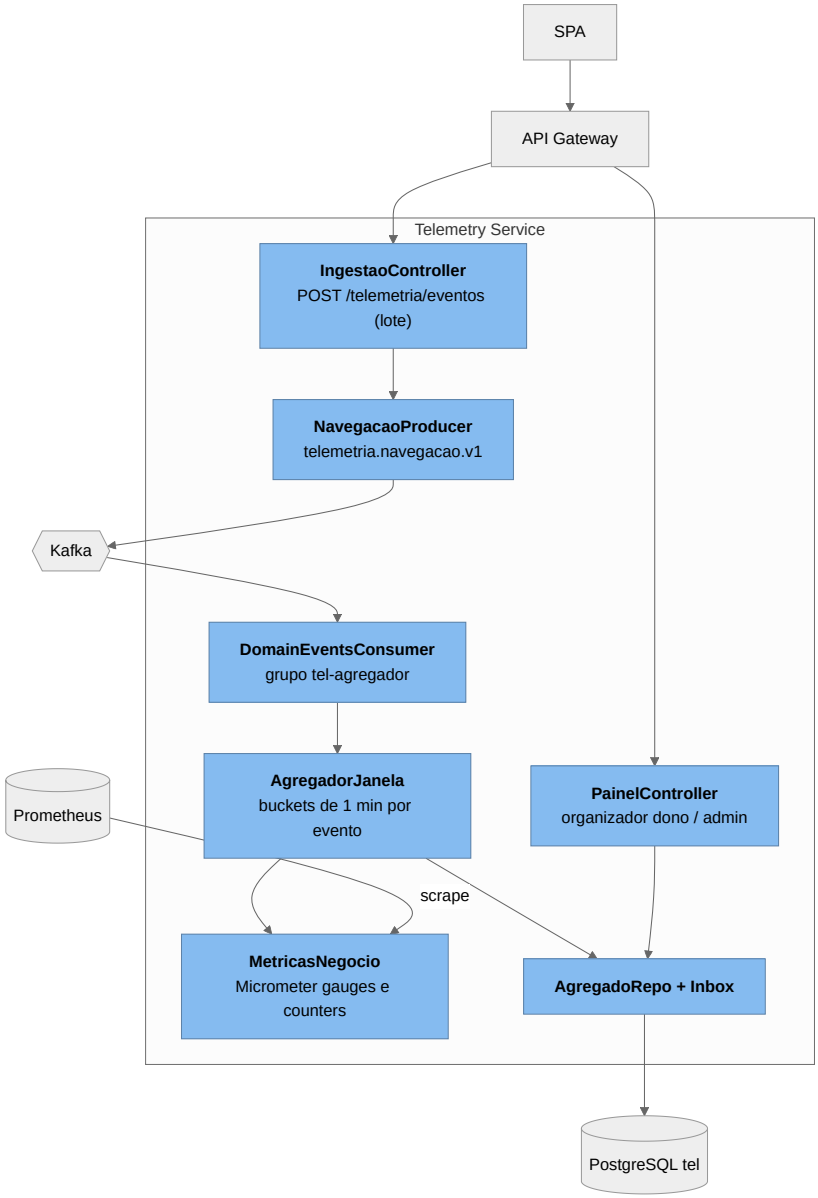


Figura 8. C3: componentes do Telemetry Service.

O Telemetry Service consome os tópicos de domínio e o de navegação no grupo `tel-agregador`, separado dos grupos de Checkout e Ingressos, de modo que todos recebem os mesmos eventos sem competir (documento 3). O painel do organizador filtra pelos eventos do organizador usando a projeção `evento → organizador` recebida de `inventario.catalogo.v1`.

1.5 Fluxos críticos (diagramas de sequência)

1.5.1 Reserva concorrente do mesmo assento

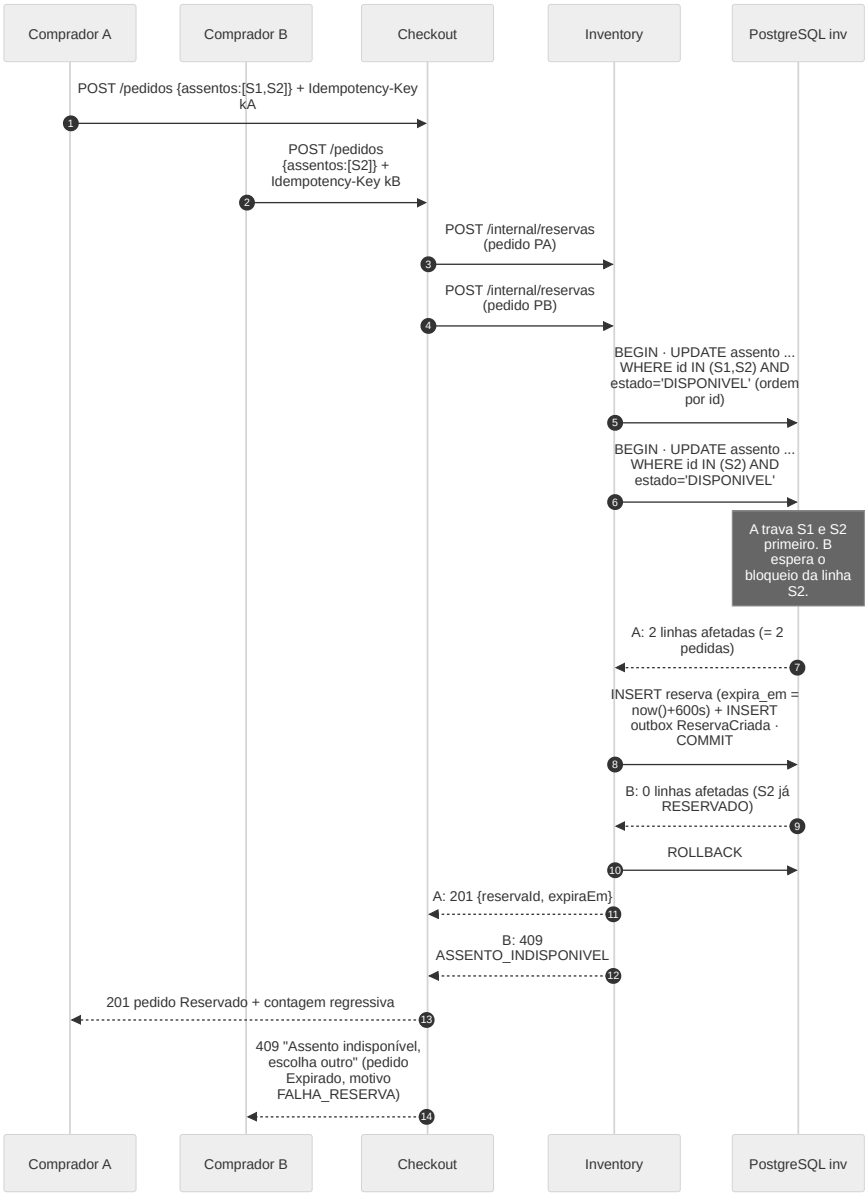


Figura 9. Dois compradores disputando S2. Com 1.000 compradores o comportamento é o mesmo: uma única transação vence (RNF01, US18).

1.5.2 Pagamento, Outbox, Kafka e emissão do ingresso

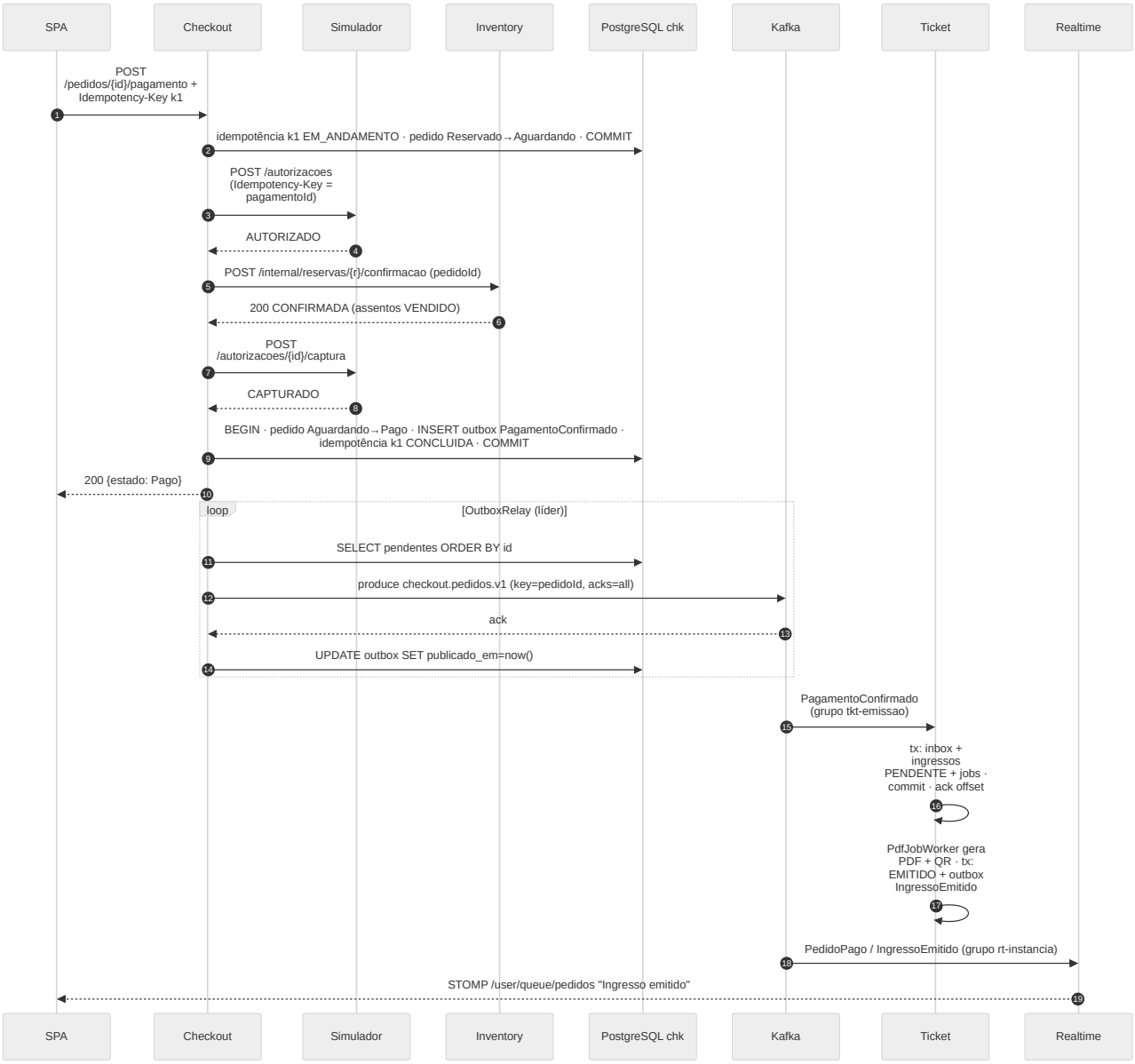


Figura 10. Caminho feliz do pagamento até o aviso em tempo real (US22, US23, US25, US28, RNF12).

1.5.3 Expiração concorrendo com a confirmação

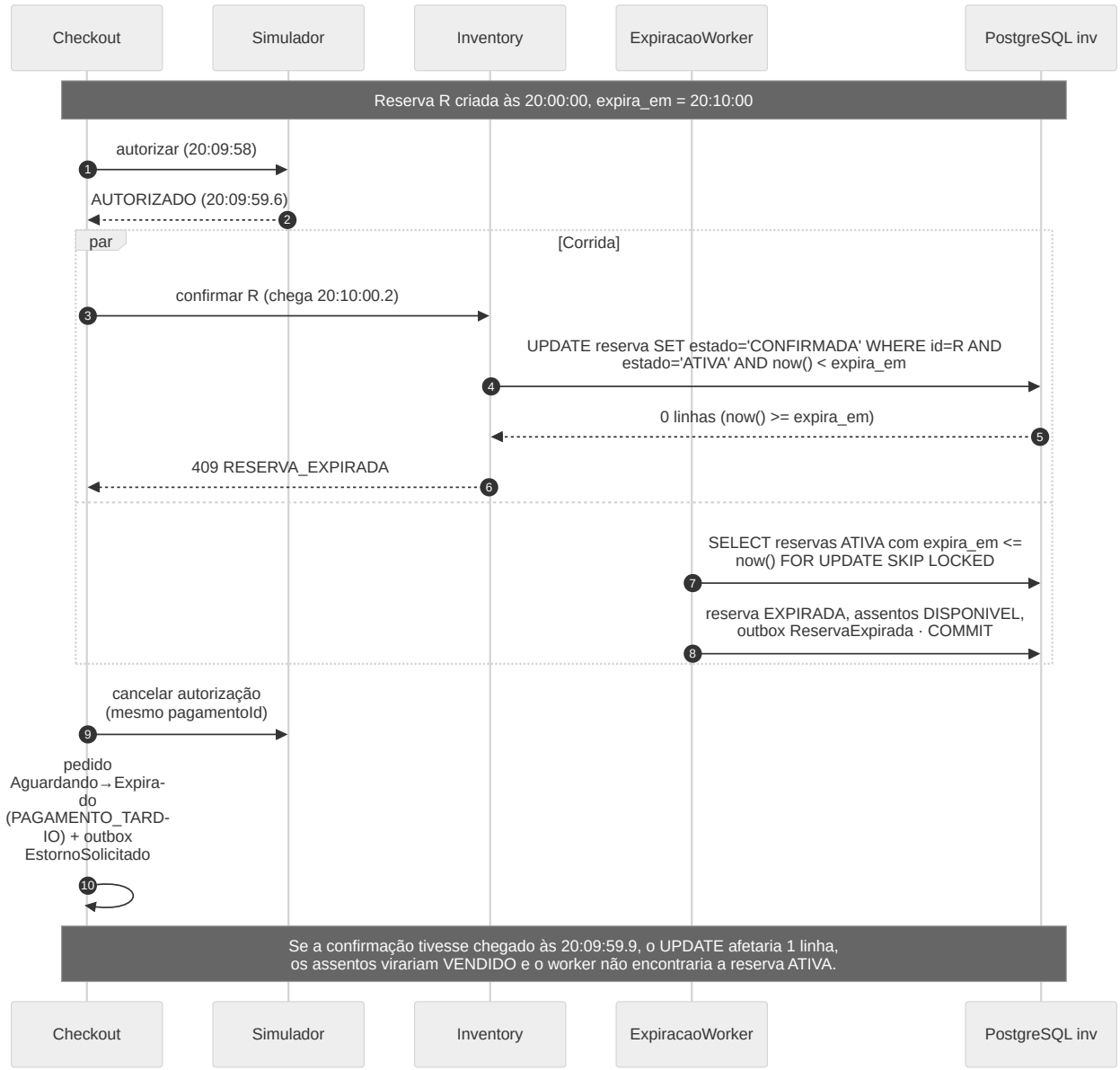


Figura 11. A linha condicional no banco do Inventário decide sozinha quem vence. Pagamento tardio nunca volta para Pago (ADR-003, US19, US21).

1.5.4 Repetição HTTP e recuperação após falha do broker

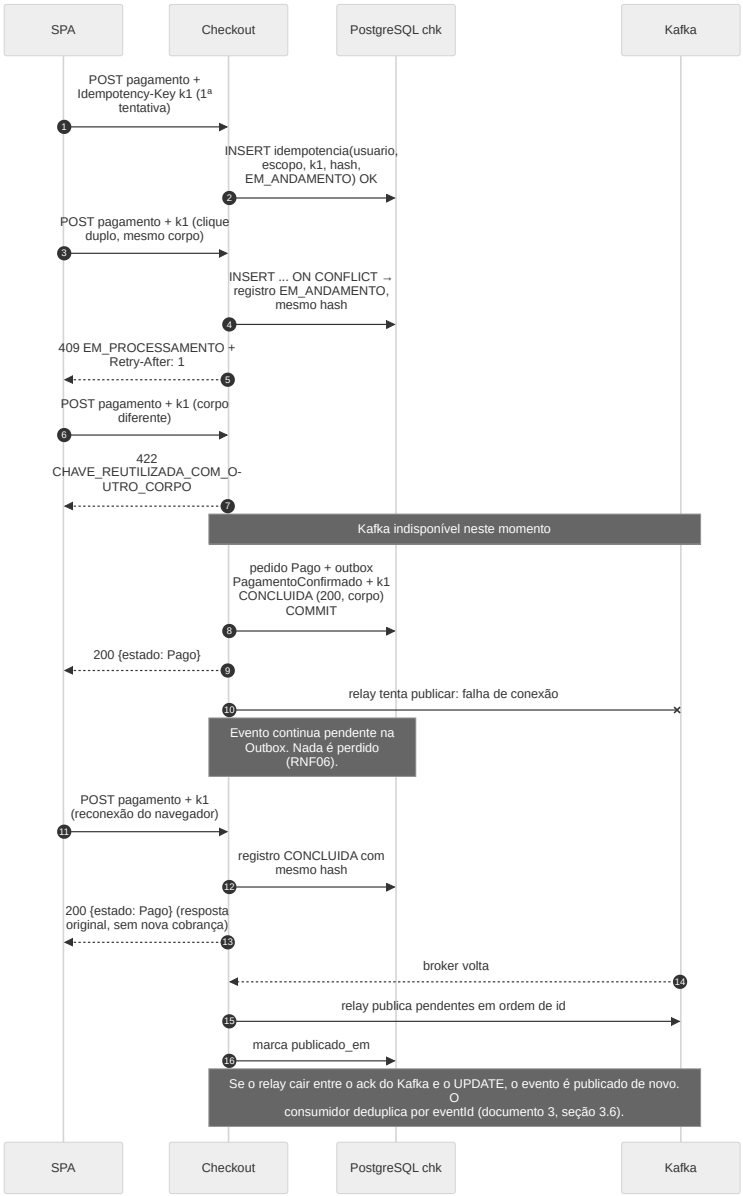


Figura 12. Idempotency-Key e Outbox sob repetição e falha do broker (RNF05, RNF06, RNF07, US22, US23).

Regras da Idempotency-Key [Proposta para revisão do grupo], detalhando RNF05 e US22:

Situação	Resposta
POST de escrita sem `Idempotency-Key`	400
Chave nova	Processa; registro passa de `EM_ANDAMENTO` para `CONCLUIDA` guardando status HTTP e corpo
Mesma chave, mesmo hash, `CONCLUIDA`	Devolve a resposta guardada, sem novo efeito
Mesma chave, mesmo hash, `EM_ANDAMENTO`	409 com `Retry-After: 1` (não executa em paralelo)
Mesma chave, hash diferente	422 (definido na US22)
`EM_ANDAMENTO` há mais de 30 s (processo caiu)	Não reexecuta às cegas: o pedido é reconciliado pelo estado atual na FSM e no simulador, e a resposta é derivada desse estado

Escopo da chave: `(usuarioId, metodo + rota, chave)`; duas pessoas podem usar o mesmo UUID sem colisão. Retenção: 24 h (RNF05), removida por job diário.

1.6 Tempo real: caminho WebSocket e controle de canais

Aspecto	Desenho **[Proposta para revisão do grupo]**
Conexão	SPA abre `wss://.../ws` no Realtime Service; o *frame* STOMP `CONNECT` leva o access token; o `ChannelInterceptor` valida assinatura, `exp` e papéis
Canais	`/topic/eventos/{eventoId}/assentos` (mapa de assentos, qualquer usuário autenticado); `/user/queue/pedidos` (pedidos e ingressos do próprio usuário, destino resolvido pelo `sub` do token); `/topic/organizador/eventos/{eventoId}/vendas` (só o organizador dono, verificado na projeção de eventos)
Autorização	Cada `SUBSCRIBE` é checado. Um cliente não consegue assinar a fila de outro usuário porque o destino `/user/...` é sempre resolvido pelo servidor a partir do token

Aspecto	Desenho **[Proposta para revisão do grupo]**
Fonte das mensagens	O Realtime não recebe chamadas dos serviços: ele consome `inventario.reservas.v1`, `checkout.pedidos.v1` e `ingressos.v1` e traduz para mensagens de tela
Múltiplas instâncias	Cada instância entra em um **consumer group exclusivo** (`rt-{instanceId}`) com `auto.offset.reset=latest`; todas recebem todos os eventos e cada uma entrega apenas às sessões conectadas nela. Não há estado compartilhado entre instâncias
Reconexão	O cliente reconecta em até 5 s (RNF29) e, ao reconectar, busca o estado atual por REST (`GET /pedidos/{id}`, mapa de assentos). O WebSocket é otimização; a fonte de verdade continua sendo a API
Expiração do token	Sessão WebSocket é encerrada quando o access token vence; a SPA renova pelo `/auth/refresh` e reconecta

1.7 Segurança transversal

Tema	Decisão
Tokens	Access token JWT RS256 do Keycloak, validade ≤ 5 min, validado pelo JWKS em cada contêiner (RNF18)
Sessão	Refresh token do Keycloak guardado cifrado no Redis pelo IAM; SPA recebe identificador opaco em cookie `HttpOnly` (ADR-004, RNF19)
RBAC	Papéis no realm; anotações `@PreAuthorize` em todo controller e checagem de dono do recurso (organizador do evento, cliente do pedido, operador vinculado) (RNF20)
Auditoria	Eventos de credencial gravados *append-only* no banco `iam` (permissão só de `INSERT` e `SELECT` para o usuário da aplicação) (RNF21)
Ingresso	QR com `ingressoId`, `eventoId` e assinatura Ed25519; chave privada só no Ticket Service via variável de ambiente (RNF22)
Segredos	Nada versionado; `.env.example` sem valores reais; gitleaks no CI (RNF23)
Logs	JSON com `traceId`; filtro mascara `Authorization`, cookies e chaves (RNF27)

1.8 Rastreabilidade

Requisito	Onde é atendido neste documento
US18, US19, RNF01, RNF02, RNF03	1.4.1, Figuras 9 e 11, ADR-002
US21, US22, US23, US24, RNF04, RNF05, RNF06, RNF15	1.4.2, Figuras 10 a 12, seção 1.5.4, ADR-003
US25, US28, US29, RNF07, RNF08, RNF12, RNF22, RNF29	1.4.3, Figura 10, seção 1.6
RNF18 a RNF21, RNF23	1.3.2, 1.4.4, 1.7, ADR-004
RNF25, RNF26, RNF28	Figura 3, 1.4.5
RNF34	Introdução da seção 1.4 (hexagonal + ArchUnit), ADR-001
RNF37 a RNF40, US32, US33	Contêineres em 1.3; detalhes no documento 4

2. Descrição do Modelo de Dados Conceitual

> Versão 1.0 · 04/11/2026 · Grupo 9 · Será atualizado nos Pacotes 3 e 4. Rótulos: **[Enunciado]** · **[Decisão do grupo, Pacote 1]** · **[Proposta para revisão do grupo]**. Modelo **conceitual**: mostra conceitos do negócio, relacionamentos, cardinalidades e regras. Tipos físicos, índices e migrações serão definidos com o código (Pacote 3).

2.1 Princípios

1. **Cada entidade tem um único serviço dono [Decisão do grupo, Pacote 1 (P03, RNF34)]**. Só o dono grava; os outros recebem cópias (projeções) por eventos ou consultam a API.
2. **Referências entre serviços são identificadores**, não chaves estrangeiras. No diagrama, linhas que cruzam serviços estão marcadas como "ref." e não implicam *join* nem FK entre bancos (ADR-001).
3. Identificadores são UUID (v7, ordenáveis por tempo), gerados pelo dono.
4. Instantes são gravados em **UTC** (`timestamptz`); a SPA apresenta em pt-BR, fuso America/Sao_Paulo (RNF44).
5. Valores monetários em **centavos inteiros** (`BIGINT`) com moeda `BRL`; nunca ponto flutuante.
6. Dados pessoais mínimos (LGPD, RNF23): senha só no Keycloak; o PulsePass guarda `sub` do Keycloak, nome de exibição e e-mail; CPF só se o titular do ingresso precisar dele na portaria (US30).

2.2 Visão geral das entidades e donos

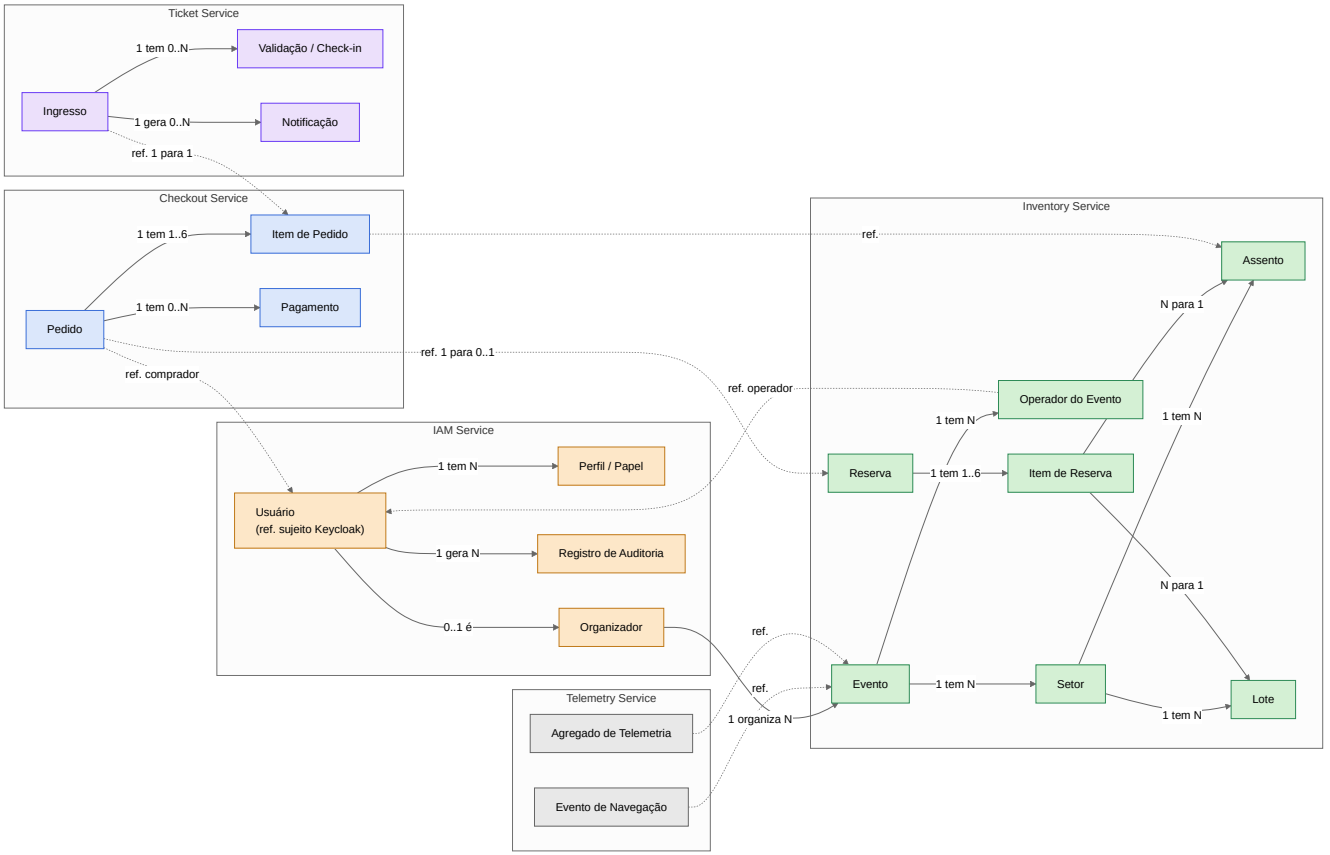


Figura 13. Entidades por serviço dono. Linhas contínuas: relacionamento dentro do mesmo banco. Linhas tracejadas "ref.": referência por identificador entre serviços.

2.3 Modelo por serviço (cardinalidades e atributos relevantes)

2.3.1 Inventory Service (banco `inv`)

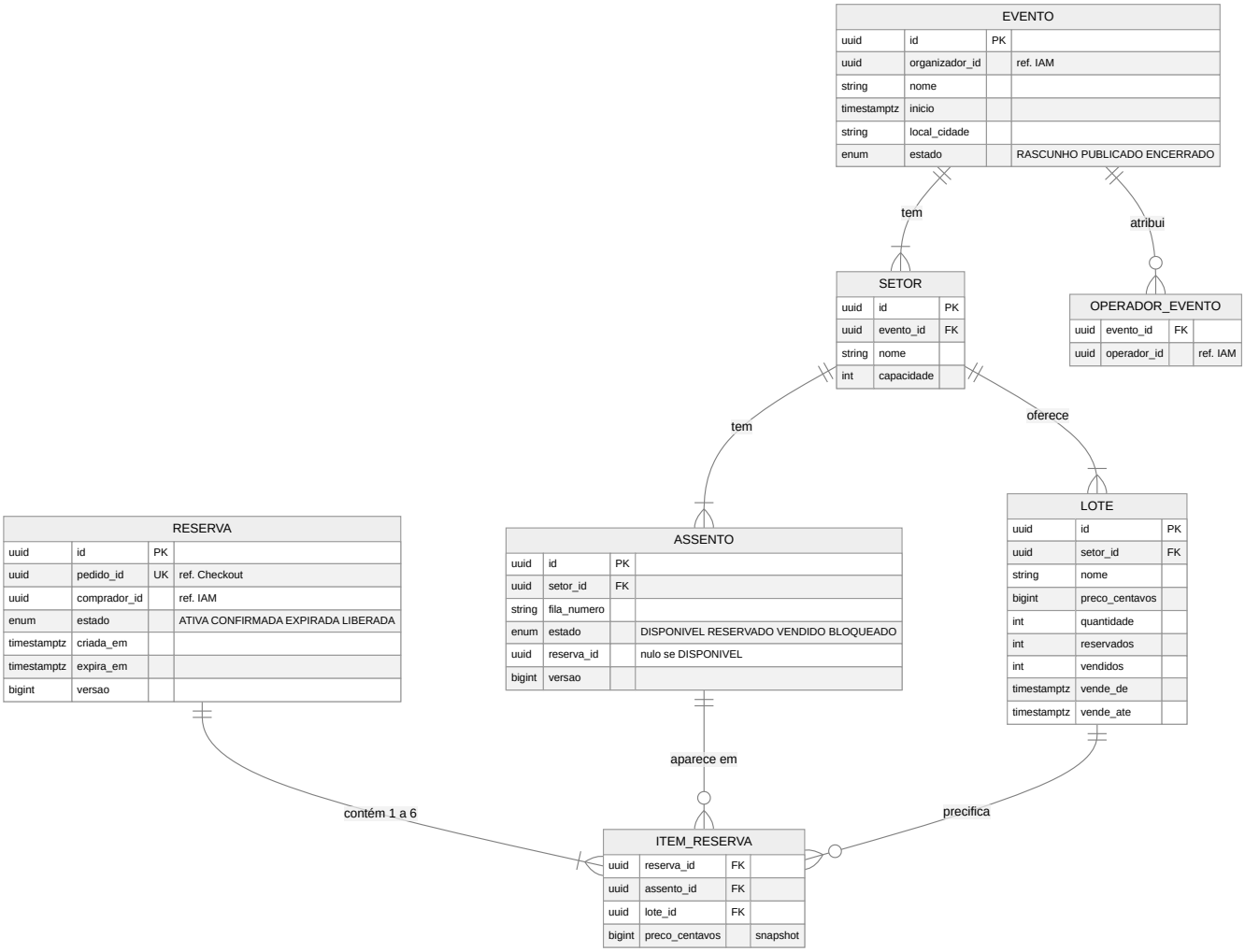


Figura 14. Inventário: catálogo, mapa, lotes e reservas.

Lote × assento [Proposta para revisão do grupo]: o enunciado pede "criação de lotes e mapas de assentos" sem dizer como se relacionam. Propomos que **um lote pertença a um setor** e define preço, quantidade e janela de vendas. Um assento não é amarrado a um lote fixo: na reserva, o assento recebe o preço do **lote ativo do seu setor** naquele instante, e o `ITEM_RESERVA` guarda o lote e o preço (*snapshot*). Isso permite "1º lote, 2º lote" no mesmo setor sem recadastrar o mapa. A quantidade do lote é respeitada na mesma transação da reserva (`UPDATE lote SET reservados = reservados + n WHERE id = ? AND reservados + vendidos + n <= quantidade`).

Operador do evento: entidade adicional, necessária porque a US29 exige que o operador só valide eventos aos quais foi vinculado. Fica no Inventário porque é o organizador dono do evento quem atribui operadores.

2.3.2 Checkout Service (banco `chk`)

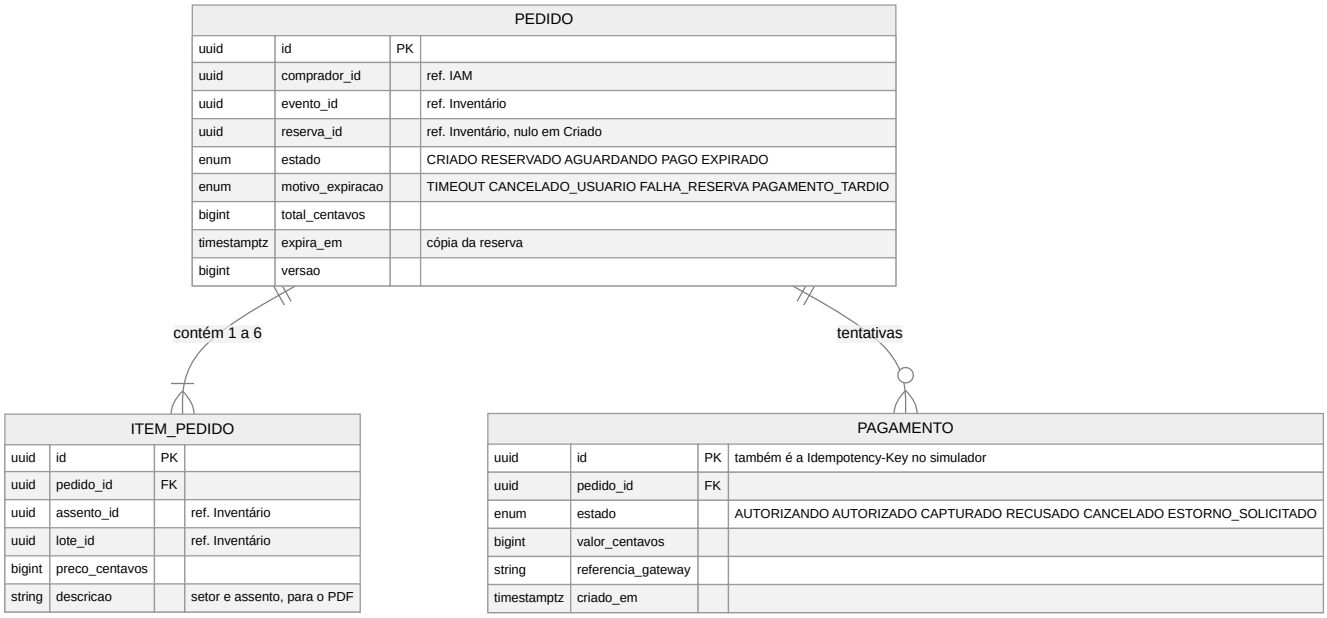


Figura 15. Checkout: pedido, itens e tentativas de pagamento.

O pedido tem **0..N pagamentos** porque, pela US24, uma recusa devolve o pedido a Reservado e o comprador pode tentar de novo dentro do prazo. No máximo um pagamento pode estar `AUTORIZADO` ou `CAPTURADO` por pedido (unicidade parcial). Nenhum dado de cartão é guardado (os dados são simulados).

2.3.3 Ticket Service (banco `tkt`)

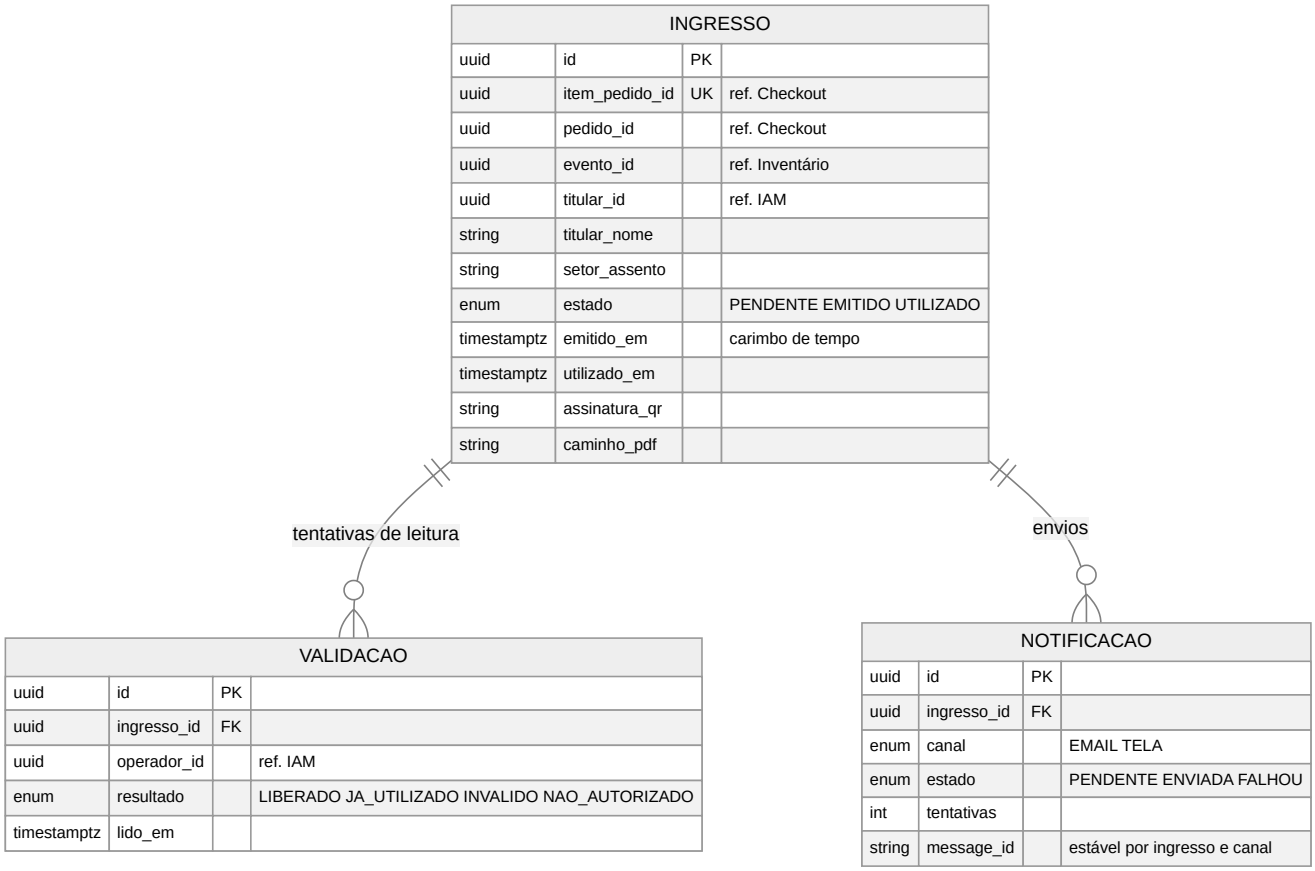


Figura 16. Ingressos: emissão, validação e notificações.

2.3.4 IAM Service (banco `iam`) e Telemetry Service (banco `tel`)

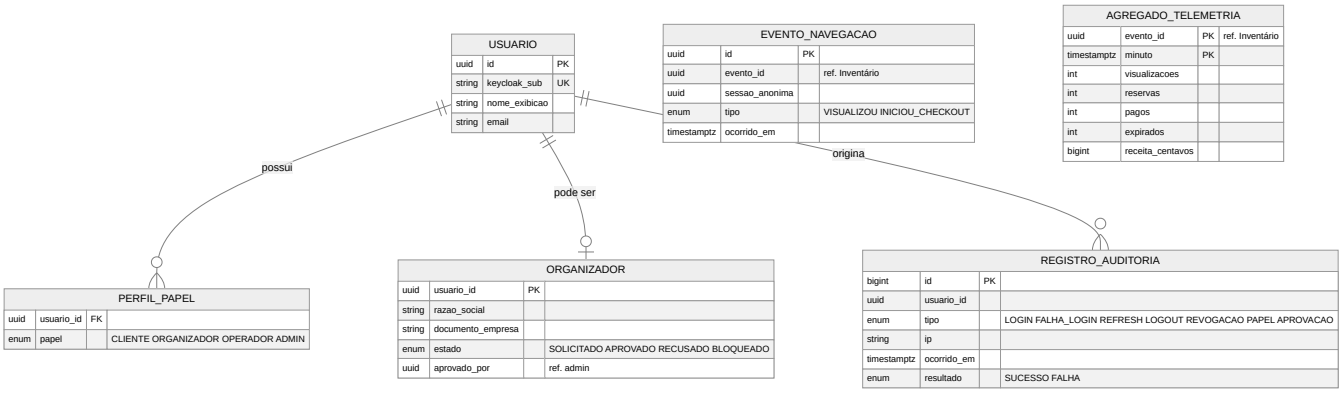


Figura 17. IAM e Telemetria. Os papéis vivem no Keycloak; `PERFIL_PAPEL` é cópia para consulta e auditoria, atualizada pelo IAM quando ele altera papéis via Admin API.

2.4 Máquina de estados do pedido

A FSM é **exigida pelo enunciado** com os cinco estados Criado, Reservado, Aguardando, Pago e Expirado. As transições abaixo são as do Pacote 1 (US21, US24); este pacote só acrescenta as pré-condições e efeitos. **Nenhum estado novo foi criado**: cancelamento, recusa definitiva e pagamento tardio são **motivos** de `Expirado`, não estados.

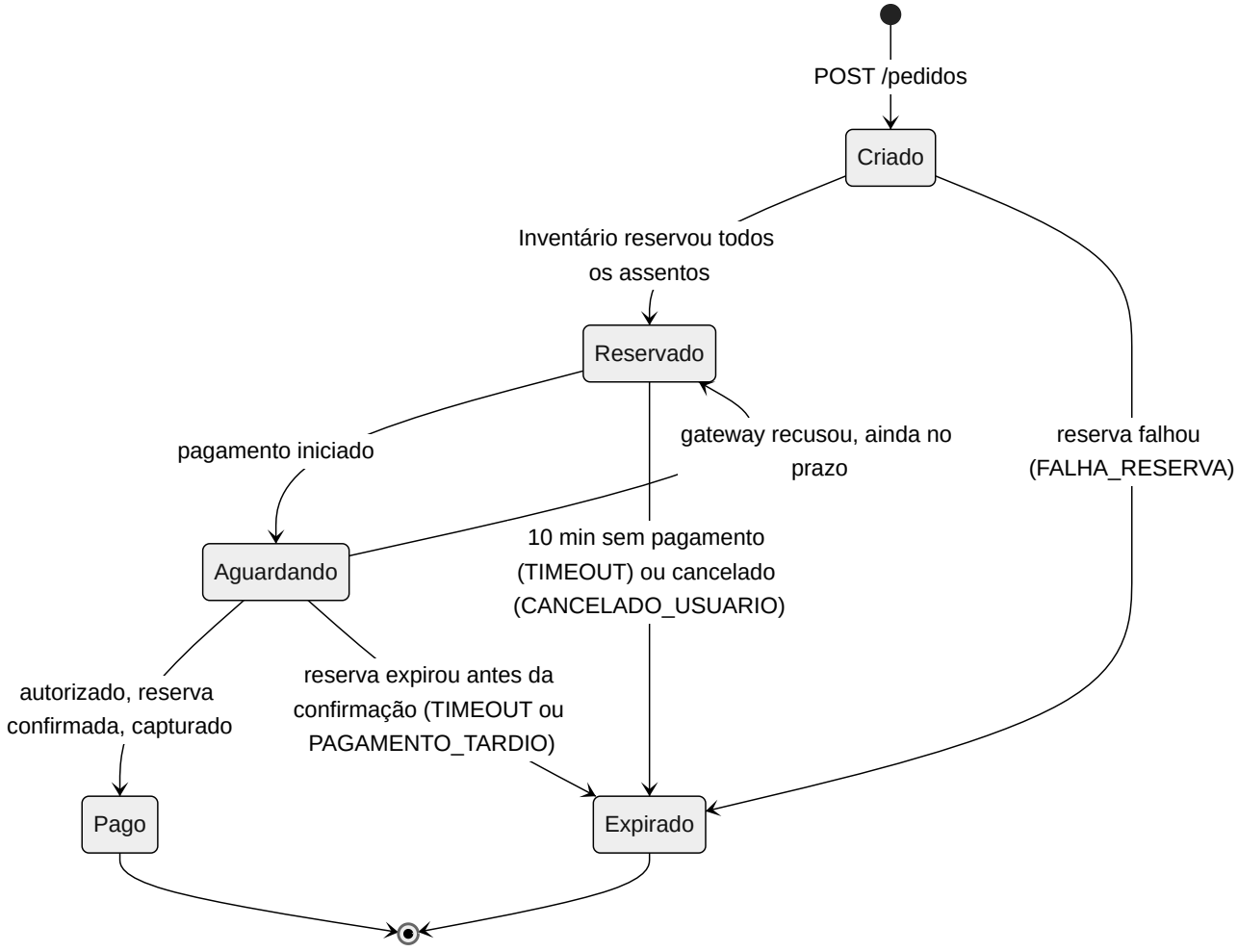


Figura 18. FSM do pedido [Enunciado + Decisão do grupo, Pacote 1 (US21, US24)].

De	Para	Gatilho	Pré-condição	Efeitos na mesma transação
-	Criado	`POST /pedidos` com Idempotency-Key	1 a 6 assentos, evento publicado, lote em janela de vendas	Insere pedido e itens; registro de idempotência
Criado	Reservado	Resposta 201 do Inventário	`reservaId` e `expiraEm` recebidos	Grava `reserva_id`, `expira_em`; Outbox `PedidoReservado`

De	Para	Gatilho	Pré-condição	Efeitos na mesma transação
Criado	Expirado	409 ou falha definitiva do Inventário	-	Motivo `FALHA_RESERVA`; Outbox `PedidoExpirado`
Reservado	Aguardando	`POST /pedidos/{id}/pagamento`	`now() < expira_em` (checagem local, o Inventário confirma depois)	Cria `PAGAMENTO AUTORIZANDO`; Outbox `PagamentoIniciado`
Reservado	Expirado	`ReservaExpirada` consumido ou cancelamento	Pedido não está Pago	Motivo `TIMEOUT` ou `CANCELADO_USUARIO`; Outbox `PedidoExpirado`
Aguardando	Pago	Autorização + confirmação no Inventário + captura	Inventário respondeu 200 (ADR-003)	Pagamento `CAPTURADO`; Outbox `PagamentoConfirmado`
Aguardando	Reservado	Simulador recusou	Reserva ainda válida	Pagamento `RECUSADO`; Outbox `PagamentoRecusado`; prazo original mantido (US24)
Aguardando	Expirado	Inventário respondeu 409 `RESERVA_EXPIRADA` ou `ReservaExpirada` consumido	Pedido não está Pago	Cancela autorização; pagamento `CANCELADO` ou `ESTORNO_SOLICITADO`; motivo `PAGAMENTO_TARDIO` se havia autorização; Outbox `PedidoExpirado` e, se for o caso, `EstornoSolicitado`
Pago, Expirado	qualquer	qualquer evento	-	**Rejeitado**, registrado em log e métrica `fsm_transicao_invalida_total` (RNF04)

Toda transição é feita por `UPDATE pedido SET estado = :novo, versao = versao + 1 WHERE id = :id AND estado = :esperado`: se outra transação mudou o estado antes, zero linhas são afetadas e a transição não acontece. Isso torna seguras as corridas entre o consumidor de expiração e o fluxo de pagamento.

2.5 Estados auxiliares (não fazem parte da FSM do pedido)

Entidade	Estados	Por que existem
Reserva (Inventário)	ATIVA → CONFIRMADA · ATIVA → EXPIRADA · ATIVA → LIBERADA	A reserva tem vida própria no Inventário; é o que o ADR-003 usa como ponto de decisão
Assento	DISPONIVEL ⇄ RESERVADO → VENDIDO; BLOQUEADO (fora de venda, definido pelo organizador)	Mapa de assentos e garantia de zero overselling
Pagamento	AUTORIZANDO → AUTORIZADO → CAPTURADO; → RECUSADO; AUTORIZADO → CANCELADO; CAPTURADO → ESTORNO_SOLICITADO	Duas fases do simulador (ADR-003)
Ingresso	PENDENTE → EMITIDO → UTILIZADO	PDF é gerado de forma assíncrona; uso único (US29)

2.6 Invariantes e como cada uma é garantida

#	Invariante	Garantia	Requisito
I1	Um assento tem no máximo uma reserva ATIVA ou CONFIRMADA	`UPDATE` condicional `estado='DISPONIVEL'` + `assento.reserva_id` único enquanto não DISPONIVEL; decisão só no PostgreSQL (ADR-002)	RNF01
I2	Reserva com vários assentos é tudo ou nada	Uma transação; linhas afetadas = assentos pedidos, senão *rollback*	RNF02
I3	Reserva só é confirmada antes de `expira_em`	`WHERE estado='ATIVA' AND now() < expira_em` no `UPDATE` de confirmação, relógio do banco	RNF03, ADR-003
I4	Só o comprador da reserva pode pagá-la	Pedido carrega `comprador_id`; reserva carrega `pedido_id` único; Checkout verifica `sub` do JWT	RNF20
I5	Assento VENDIDO nunca volta a DISPONIVEL	Nenhuma transição prevista (cancelamento pós-pago está fora de escopo na Visão 2.10)	RNF01
I6	Lote não vende acima da quantidade	`reservados + vendidos + n <= quantidade` no mesmo `UPDATE`	US10
I7	Pedido Expirado nunca vira Pago	FSM: estados finais rejeitam transições	US21
I8	Um ingresso por item de pedido	`UNIQUE (item_pedido_id)` no Ticket + Inbox por `eventId`	RNF07
I9	Ingresso é usado uma vez	`UPDATE ingresso SET estado='UTILIZADO' WHERE id=? AND estado='EMITIDO'`	RNF22, US29
I10	QR não pode ser forjado	Assinatura Ed25519 sobre `ingressoId` + `eventoId` + `emitidoEm`	RNF22
I11	Auditoria não é alterada	Usuário da aplicação só tem `INSERT/SELECT` em `registro_auditoria`	RNF21

A unicidade do ingresso (I8) é **defesa em profundidade**, não a garantia contra overselling: a garantia é I1, no momento da reserva.

2.7 Complemento técnico (não faz parte do modelo de negócio)

Estas tabelas existem para resolver concorrência, escrita dupla e idempotência. Cada serviço tem a sua cópia no próprio banco.

Tabela	Serviços	Campos principais	Para que serve	Retenção
`outbox`	IAM, INV, CHK, TKT	`id` (sequencial), `event_id`, `topico`, `chave`, `tipo`, `payload` JSON, `criado_em`, `publicado_em`, `tentativas`	Transactional Outbox (RNF06, ADR-005)	Registros publicados apagados após 7 dias
`mensagem_processada` (Inbox)	INV, CHK, TKT, TEL	`grupo`, `event_id` (PK composta), `processado_em`	Deduplicação no consumo (RNF07)	14 dias (maior que a retenção dos tópicos, 7 dias)
`idempotencia`	CHK	`usuario_id`, `escopo`, `chave` (único), `hash_requisicao`, `estado`, `status_http`, `corpo_resposta`, `expira_em`	Idempotency-Key (RNF05, seção 1.5.4)	24 h
`job_pdf`	TKT	`ingresso_id` (PK), `estado`, `tentativas`, `ultimo_erro`	Geração assíncrona e repetível do PDF	Até o ingresso ser EMITIDO + 30 dias
Chaves Redis `iam:sessao:*`, `iam:sessao-usada:*`, `iam:revogado:*`	IAM, Gateway	Refresh token cifrado, marcador de uso, `sid` revogado	Sessões e revogação (ADR-004, RNF19)	TTL 8 h / vida do access token
Chaves Redis `inv:catalogo:*`, `inv:mapa:*`	INV	Projeções de leitura	Cache do catálogo (RNF09)	TTL 30 s / 2 s

2.8 Retenção e dados pessoais

Dado	Retenção proposta	Motivo
Pedidos, pagamentos, ingressos	Durante o semestre (ambiente acadêmico)	Demonstração; em produção seguiria obrigação fiscal
Auditoria de credenciais	≥ 90 dias (RNF21)	Requisito do grupo
Eventos de navegação brutos	7 dias, depois só agregados	Minimização (LGPD)
Logs	7 dias no ambiente local	Sem dados sensíveis (RNF27)

3. Descrição da Estrutura de Mensageria do Kafka

> Versão 1.0 · 04/11/2026 · Grupo 9 · Será atualizado nos Pacotes 3 e 4. Rótulos: **[Enunciado]** · **[Decisão do grupo, Pacote 1]** · **[Proposta para revisão do grupo]**. Números de partições, retenções e nomes de tópicos abaixo são **propostas do grupo**, não exigências do professor. O enunciado exige Kafka como provedor de streaming e o tratamento do dilema da escrita dupla e da idempotência no consumo.

3.1 Visão geral

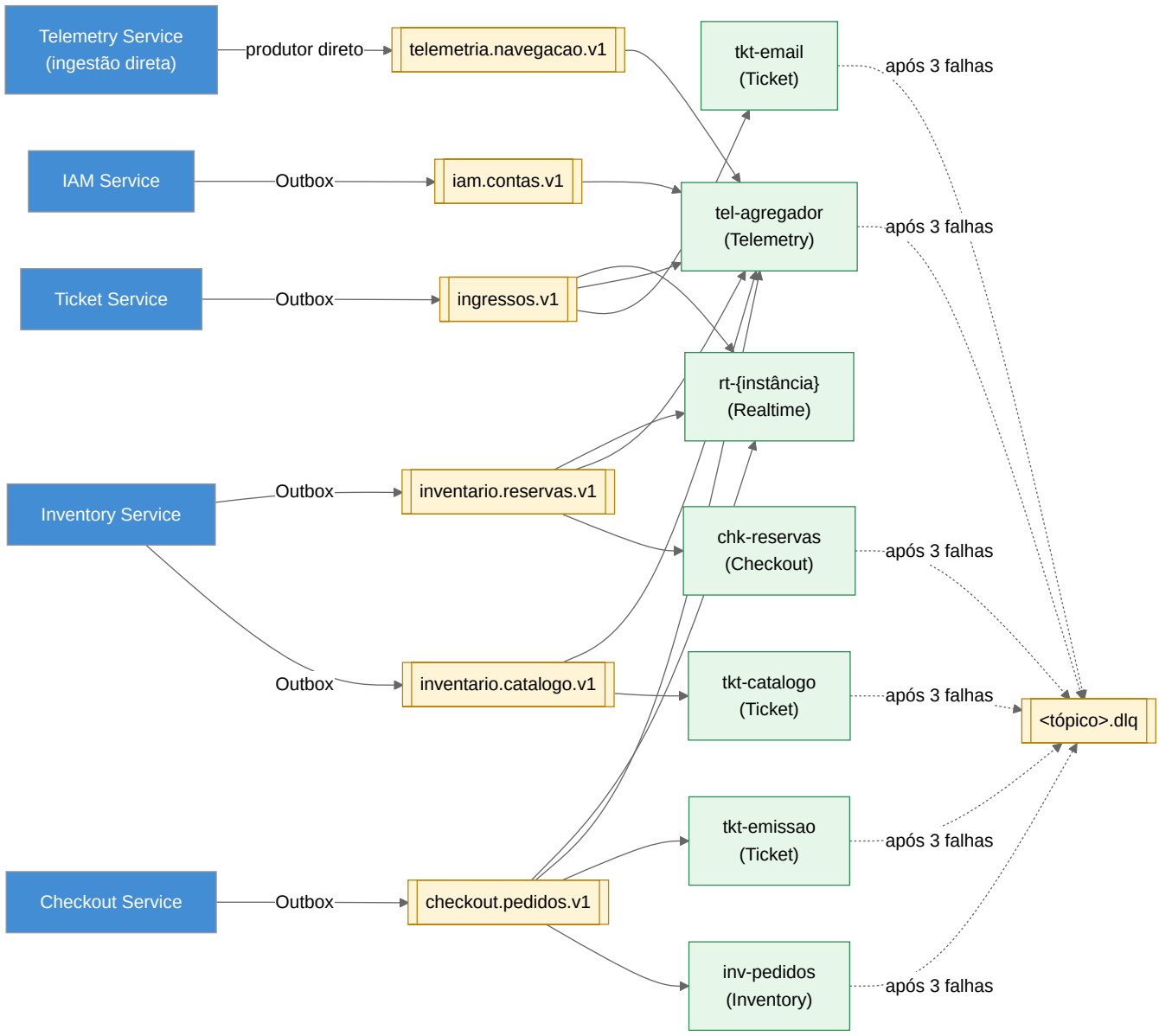


Figura 19. Tópicos, produtores e consumer groups. Cada grupo recebe todas as mensagens dos tópicos que assina; consumidores do mesmo grupo dividem as partições.

3.2 Catálogo de tópicos

Convenção de nome: `<contexto>.<agregado>.v<versão-major>` **[Proposta para revisão do grupo]**. Um tópico por agregado mantém em ordem todos os eventos da mesma reserva ou do mesmo pedido (ADR-005). O nome `inventario.reserva-expirada.v1`, citado como exemplo na spec da US18 do Pacote 1, passa a ser o tipo `ReservaExpirada` dentro de `inventario.reservas.v1`; o nome `PagamentoConfirmado` do Pacote 1 é mantido como tipo de evento.

Tópico	Tipos de evento	Produtor	Consumer groups	Chave	Ordem necessária	Partições	Retenção	DLQ
`iam.contas.v1`	`OrganizadorAprovado`, `OrganizadorBloqueado`, `PapeisAlterados`	IAM (Outbox)	`tel-agregador`	`usuarioId`	Por usuário	3	7 dias	`iam.contas.v1.dlq`

Tópico	Tipos de evento	Produtor	Consumer groups	Chave	Ordem necessária	Partições	Retenção	DLQ
`inventario.catalogo.v1`	`EventoPublicado`, `EventoEncerrado`, `LoteAlterado`, `OperadoresAtribuidos`	Inventory (Outbox)	`tkt-catalogo`, `tel-agregador`	`eventoId`	Por evento	3	7 dias	`inventario.catalogo.v1.dlq`
`inventario.reservas.v1`	`ReservaCriada`, `ReservaConfirmada`, `ReservaExpirada`, `ReservaLiberada`	Inventory (Outbox)	`chk-reservas`, `tel-agregador`, `rt-*`	`reservaId`	Por reserva	6	7 dias	`inventario.reservas.v1.dlq`
`checkout.pedidos.v1`	`PedidoReservado`, `PagamentoIniciado`, `PagamentoRecusado`, `PagamentoConfirmado`, `PedidoExpirado`, `EstornoSolicitado`	Checkout (Outbox)	`tkt-emissao`, `inv-pedidos`, `tel-agregador`, `rt-*`	`pedidoId`	Por pedido	6	7 dias	`checkout.pedidos.v1.dlq`
`ingressos.v1`	`IngressoEmitido`, `IngressoUtilizado`	Ticket (Outbox)	`tkt-email`, `tel-agregador`, `rt-*`	`ingressoId`	Por ingresso	3	7 dias	`ingressos.v1.dlq`
`telemetria.navegacao.v1`	`EventoVisualizado`, `CheckoutIniciado`	Telemetry (ingestão HTTP)	`tel-agregador`	`eventoId`	Não (contagem)	3	3 dias	`telemetria.navegacao.v1.dlq`
`<tópico>.dlq`	Mensagem original + cabeçalhos de erro	Qualquer consumidor	Inspeção manual (Kafka UI) e ferramenta de reprocessamento	Mesma do original	-	1	14 dias	-

Configuração comum:

Item	Ambiente local (Compose)	Observação para produção
Brokers	1 broker KRaft	3 brokers
Fator de replicação	1	3, com `min.insync.replicas=2`
Produtor	`acks=all`, `enable.idempotence=true`, `max.in.flight.requests.per.connection=5`	Igual (mantém ordem por partição com idempotência)
Consumidor	`enable.auto.commit=false`, `isolation.level=read_committed`, `max.poll.records=50`	Igual
Criação dos tópicos	Contêiner `kafka-init` cria tópicos e partições no `docker compose up` (`auto.create.topics.enable=false`)	Infra como código

O mínimo de 3 partições atende RNF13. Os tópicos de reserva e pedido têm 6 partições porque concentram o pico da abertura de vendas.

3.2.1 Por que `telemetria.navegacao.v1` não usa Outbox

A navegação vem da SPA, não de uma mudança de estado no banco. O Telemetry Service recebe o lote por HTTP e publica direto no Kafka com produtor idempotente. Perder um evento de navegação em uma queda do broker afeta só a métrica do funil, não dinheiro nem assentos. **[Proposta para revisão do grupo]**

3.3 Envelope dos eventos

Todos os eventos de domínio usam o mesmo envelope JSON **[Proposta para revisão do grupo, detalhando RNF36]**:

```
{
  "eventId": "0193f6a2-6b1e-7c11-9a4e-5d2f8e7a1b30",
  "tipo": "PagamentoConfirmado",
  "schemaVersion": 1,
  "ocorridoEm": "2026-11-04T23:09:58.412Z",
  "produtor": "checkout-service",
  "agregadoId": "0193f6a0-11aa-7d2e-8b55-0f3c2a9e4d71",
  "agregadoVersao": 4,
  "correlationId": "0193f6a0-11aa-7d2e-8b55-0f3c2a9e4d71",
  "causationId": "0193f6a2-5f00-7a00-8000-3e21c9d0aa10",
  "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "dados": {
    "pedidoId": "0193f6a0-11aa-7d2e-8b55-0f3c2a9e4d71",
    "reservaId": "0193f6a0-2c4d-7e8f-9a01-b2c3d4e5f607",
    "compradorId": "6f1c2d3e-4b5a-4c6d-8e7f-90a1b2c3d4e5",
    "eventoId": "0193e0aa-0000-7000-8000-000000000001",
    "itens": [
      { "itemPedidoId": "0193f6a0-3333-7000-8000-000000000001", "assentoId": "0193e0ab-0000-7000-8000-0000000000a1", "descricao": "Pista
A-12", "precoCentavos": 25000 }
    ],
    "totalCentavos": 25000,
    "pagoEm": "2026-11-04T23:09:58.398Z"
  }
}
```

Campo	Uso
`eventId`	UUID v7 gerado na escrita da Outbox; chave de deduplicação dos consumidores (RNF07)
`tipo` e `schemaVersion`	Seleção do *handler* e da versão do esquema
`ocorridoEm`	Instante UTC do fato (não do envio)
`agregadoId` e `agregadoVersao`	Versão monotônica do agregado; o consumidor descarta versão menor ou igual à última aplicada
`correlationId`	Liga tudo o que nasceu do mesmo pedido
`causationId`	`eventId` ou requisição que causou este evento
`traceparent`	Também enviado como cabeçalho Kafka para o OpenTelemetry (RNF26)

Nunca vão no evento: JWT, refresh token, cookie, dados de cartão, CPF completo, chave de assinatura do QR. O evento leva identificadores; quem precisa de mais dados consulta o dono.

Os exemplos de cada tipo estão em `pacote-2/contratos/` (arquivos JSON). Os JSON Schemas formais serão gerados no Pacote 3.

3.3.1 Evolução dos esquemas

Mudança	Permitida em v1?	Como
Adicionar campo opcional	Sim	Consumidores ignoram campos desconhecidos (Jackson `FAIL_ON_UNKNOWN_PROPERTIES=false`)
Adicionar tipo de evento novo no tópico	Sim	Consumidor que não conhece o tipo faz *skip* com log e confirma o offset
Renomear ou remover campo, mudar tipo ou significado	Não	Novo tópico `...v2`; produtor publica nos dois durante a transição
Mudar a chave de partição	Não	Novo tópico `...v2`

Testes de contrato em cada consumidor carregam os exemplos de `contratos/` e verificam que o *handler* aceita todas as versões ativas (RNF36).

3.4 Transactional Outbox (escrita dupla)

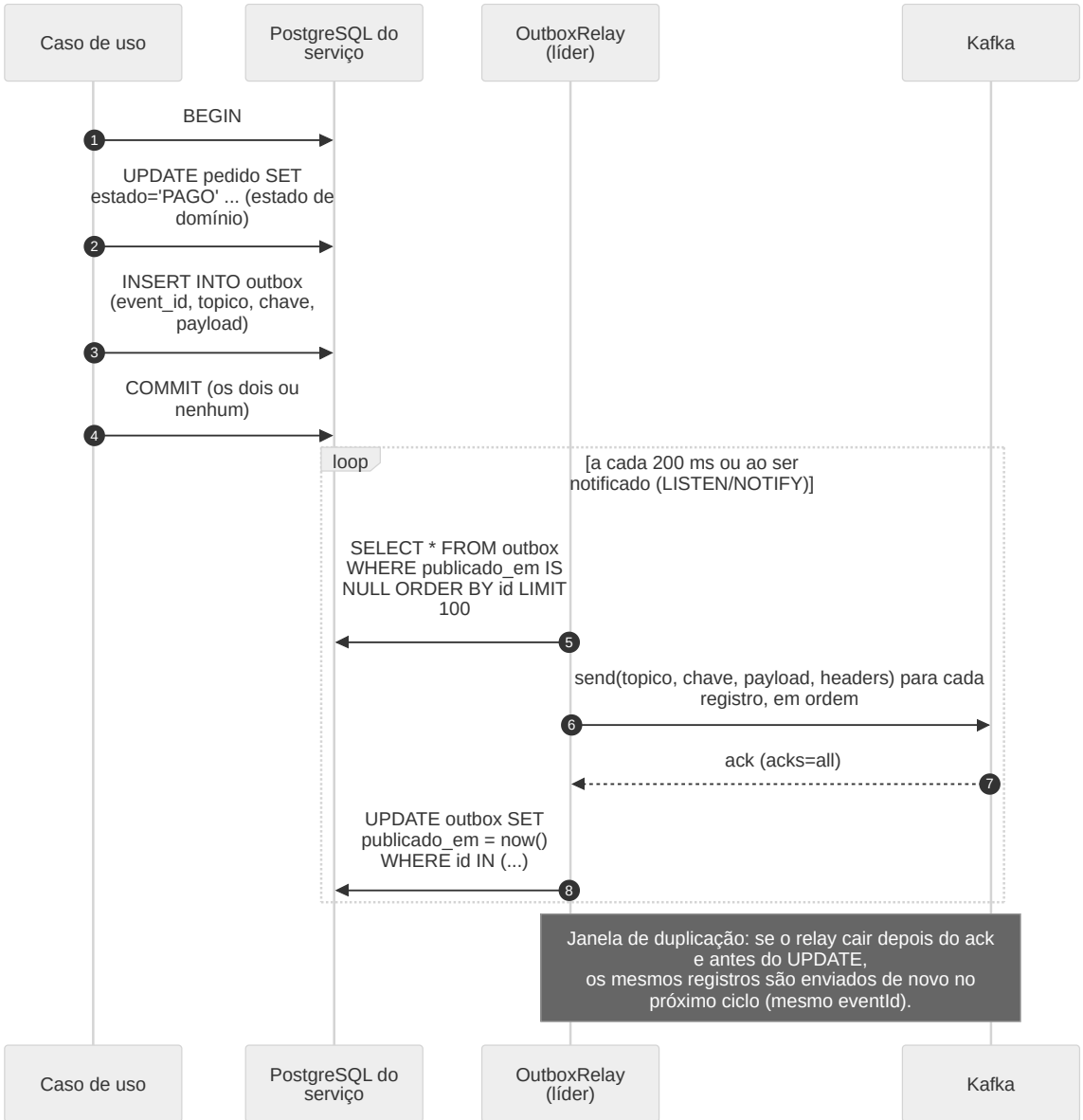


Figura 20. Outbox: o estado e o evento são gravados juntos; o envio ao Kafka acontece depois (RNF06, US23).

Regra	Detalhe
Sem escrita direta no Kafka em caso de uso	ArchUnit proíbe `KafkaTemplate` fora do pacote `adapters.outbox` (RNF06)
Um relay ativo por serviço	Cada réplica tenta `pg_try_advisory_lock(<id do serviço>)`; só quem obtém o lock publica. Se a réplica cair, o lock é liberado com a conexão e outra assume em segundos
Ordem	O relay lê por `id` crescente e, se um envio falha, para o lote naquele registro e tenta de novo; registros posteriores não passam na frente
Broker fora	Registros ficam pendentes; o relay tenta com *backoff* até 30 s; métrica `outbox_pendentes` e alerta se > 1.000 por 1 min
Limpeza	Job diário apaga publicados há mais de 7 dias
Alternativa futura	Debezium lendo a tabela `outbox` (CDC), mantendo o mesmo formato de evento

3.5 Semântica de entrega

O PulsePass trabalha com **entrega pelo menos uma vez** e **efeitos idempotentes**. Não prometemos "exatamente uma vez" ponta a ponta, porque o caminho cruza PostgreSQL, Kafka, sistema de arquivos e SMTP, e as transações do Kafka só cobrem o próprio Kafka.

Origem de duplicata	Onde é absorvida
Relay republica após falha	`mensagem_processada (grupo, eventId)` no consumidor
Consumidor processou e caiu antes de confirmar o offset	Mesmo mecanismo: na releitura o `eventId` já está na Inbox
Rebalanceamento do grupo	Idem
Reprocessamento manual da DLQ ou *replay* do tópico	Idem; o reprocessamento nunca gera `eventId` novo

3.6 Consumo idempotente

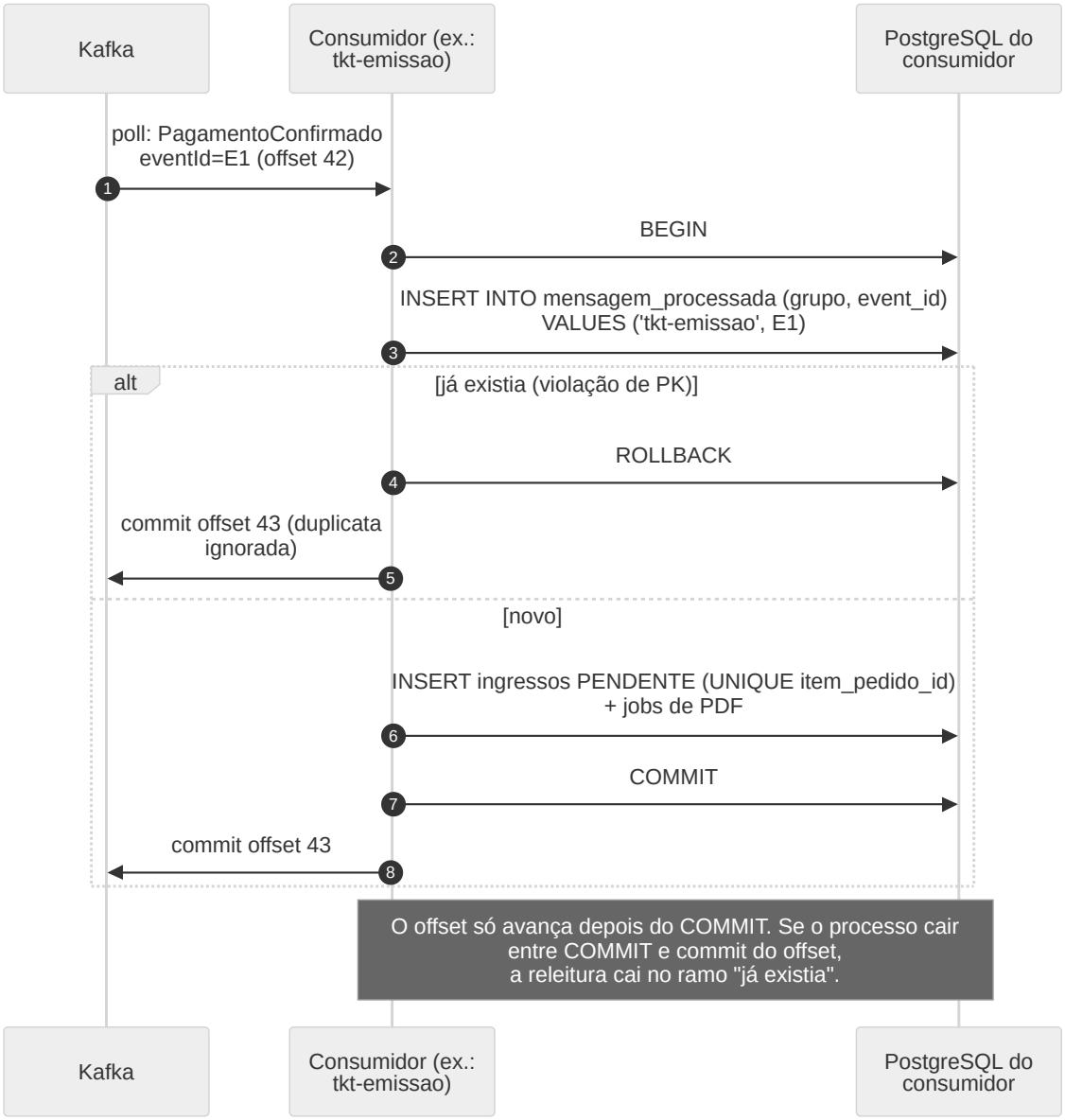


Figura 21. Deduplicação na mesma transação do efeito (RNF07, US25).

Regras:

1. **A Inbox e o efeito são gravados na mesma transação.** O `eventId` nunca é marcado como processado antes do efeito persistir, então uma falha no meio não descarta o evento.
2. **Transições guardadas por estado e versão.** Além da Inbox, todo efeito é condicional (`WHERE estado = :esperado`, `agregadoVersao > ultimaAplicada`). Um evento antigo que chega depois de um mais novo vira no-op registrado, não regressão de estado.
3. **Efeitos externos** (PDF, e-mail) não entram na transação. Eles viram *jobs* com identificador estável (seção 3.7).

3.7 PDF e notificações: falhas parciais

Passo	Identificador estável	Se falhar no meio
Criar ingressos PENDENTE + `job_pdf`	`item_pedido_id` (único)	Transação inteira desfeita; evento relido
Gerar PDF e gravar em `ingressos/{ingressoId}.pdf`	`ingressoId`	Arquivo é sobrescrito na nova tentativa (gravação em arquivo temporário + `rename` atômico)
Marcar EMITIDO + Outbox `IngressoEmitido`	`ingressoId`	Só ocorre depois do arquivo existir; em falha o job repete
Enviar e-mail (`tkt-email`)	`Message-ID: <ingressoId.email@pulsepass.local>`	`NOTIFICACAO` registra `PENDENTE` antes do SMTP e `ENVIADA` depois

Limite de idempotência do provedor externo: se o processo cair depois que o servidor SMTP aceitou a mensagem e antes de gravar `ENVIADA`, o e-mail é reenviado. O SMTP não deduplica; o `Message-ID` estável permite que clientes de e-mail agrupem a duplicata. Aceitamos esse risco: **um e-mail repetido é aceitável, um ingresso repetido não**. O mesmo vale para o simulador de pagamento, com a diferença de que o simulador **deduplica** pela `Idempotency-Key = pagamentoId` (ADR-003).

3.8 Retries, DLQ e ordem

Etapa	Comportamento **[Decisão do grupo, Pacote 1 (RNF08)] + detalhes propostos**
Erro transitório (banco indisponível, timeout)	Até 3 tentativas no próprio consumidor, *backoff* 1 s, 2 s, 4 s (`DefaultErrorHandler` do Spring Kafka). A partição fica parada durante essas tentativas, o que **preserva a ordem**
Erro permanente (JSON inválido, esquema desconhecido, violação de regra)	Vai direto para a DLQ, sem tentativas (classificado como não repetível)
Após 3 tentativas	Mensagem publicada em `<tópico>.dlq` com cabeçalhos `x-erro-classe`, `x-erro-mensagem`, `x-grupo`, `x-topico-original`, `x-particao`, `x-offset`, `x-tentativas`; offset original confirmado; consumo continua
Alerta	Métrica `kafka_dlq_mensagens_total` por tópico; alerta no Grafana se > 0 em 5 min
Inspeção	Kafka UI mostra a mensagem e os cabeçalhos
Reprocessamento	Ferramenta `dlq-replay` (comando do Telemetry/DevOps) republica a mensagem original no tópico de origem **com o mesmo `eventId`**; a Inbox e as transições guardadas garantem idempotência

Impacto na ordem. Quando uma mensagem do agregado X vai para a DLQ, as mensagens seguintes do mesmo X continuam sendo processadas e podem chegar antes dela no reprocessamento. Isso é seguro porque os efeitos são transições guardadas por estado e versão: por exemplo, se `PedidoExpirado` (versão 3) já foi aplicado, um `PagamentoIniciado` (versão 2) reprocessado depois é ignorado. Não dependemos de ordem global entre tópicos: o Checkout trata `ReservaExpirada` sem saber se o Inventário já viu `PagamentoIniciado`, porque a decisão de venda é síncrona no Inventário (ADR-003).

3.9 Isolamento dos consumer groups

Grupo	Serviço	Tópicos	Efeito	Escala
`chk-reservas`	Checkout	`inventario.reservas.v1`	`ReservaExpirada` → pedido Expirado	Até 6 réplicas (partições)
`inv-pedidos`	Inventory	`checkout.pedidos.v1`	`PedidoExpirado` → libera reserva (rede de segurança se a chamada síncrona falhou)	Até 6
`tkt-emissao`	Ticket	`checkout.pedidos.v1`	`PagamentoConfirmado` → ingressos	Até 6
`tkt-catalogo`	Ticket	`inventario.catalogo.v1`	Projeção de eventos e operadores vinculados	Até 3
`tkt-email`	Ticket	`ingressos.v1`	`IngressoEmitido` → e-mail	Até 3
`tel-agregador`	Telemetry	Todos	Agregados de funil e vendas	Até 3
`rt-{instância}`	Realtime	`inventario.reservas.v1`, `checkout.pedidos.v1`, `ingressos.v1`	Mensagens de tela; grupo exclusivo por instância, `auto.offset.reset=latest`, sem Inbox (mensagens de tela são efêmeras)	1 consumidor por instância

Ingressos e Telemetria leem `checkout.pedidos.v1` em grupos diferentes, então **cada um recebe todas as mensagens**; não competem. Dentro de um grupo, réplicas dividem as partições.

3.10 Cenários de falha

Cenário	O que acontece	Resultado
Kafka indisponível durante o checkout	Pedido vai a Pago normalmente; evento fica na Outbox; relay tenta com *backoff*	Ingresso emitido quando o broker volta; nada perdido (RNF06, CS3)
Mensagem duplicada	Inbox encontra `(grupo, eventId)`	Ignorada, offset confirmado (RNF07)
Mensagem fora de ordem	Versão do agregado menor ou estado não esperado	No-op registrado; métrica `evento_desatualizado_total`
Mensagem malformada	Falha de desserialização	Direto para DLQ, consumo continua (RNF08)

Cenário	O que acontece	Resultado
Replay do tópico inteiro	Todos os `eventId` já estão na Inbox (retenção 14 dias > 7 dias do tópico)	Nenhum efeito duplicado; teste do RNF07 reprocessa o tópico
Reinício do consumidor	Retoma do último offset confirmado; pode reler até `max.poll.records` mensagens	Inbox absorve (RNF16)
Erro depois de efeito parcial (ex.: PDF gerado, banco caiu antes de EMITIDO)	Job repete; arquivo sobrescrito; mesmo `ingressoId`	Um ingresso, um arquivo
Relay cai entre ack e `publicado_em`	Republica no próximo ciclo	Duplicata absorvida pelo consumidor
`ReservaExpirada` chega depois de `PagamentoConfirmado` do mesmo pedido	Checkout: pedido já Pago, transição inválida	Ignorado e registrado (US21); o Inventário nunca emitiria `ReservaExpirada` para reserva CONFIRMADA

3.11 Rastreabilidade

Requisito	Seção
Dilema da escrita dupla [Enunciado], RNF06, US23	3.4
Idempotência no consumo [Enunciado], RNF07, US25	3.5, 3.6, 3.7
RNF08, US25 (DLQ após 3 falhas)	3.8
RNF13 (≥ 3 partições, escala por partição)	3.2, 3.9
RNF26 (trace em cabeçalhos Kafka)	3.3
RNF28 (ingestão de navegação e compras)	3.2.1, 3.9
RNF36 (esquema versionado)	3.3, 3.3.1
US19 (`ReservaExpirada` publicado)	3.2, 3.10

4. Descrição Esquemática da Pipeline de Desenvolvimento

> Versão 1.0 · 04/11/2026 · Grupo 9 · Será atualizado nos Pacotes 3 e 4. Rótulos: **[Enunciado]** · **[Decisão do grupo, Pacote 1]** · **[Proposta para revisão do grupo]**. **Estado real em 04/11/2026:** o repositório tem só documentação. Não existem ainda código dos serviços, `docker-compose.yml`, `.github/workflows/` nem branch `develop`. Tudo neste documento que fala em executar é **planejado**; os comandos aparecem como **comandos previstos**, não como resultados.

4.1 Exigência e escopo

O enunciado exige **[Enunciado]**:

- pipeline de CI no GitHub Actions ou GitLab executando **linters, testes e compilação a cada Pull Request ou Merge na branch `develop`**;
- testes unitários cobrindo máquinas de estado com **≥ 70% de cobertura de domínio**;
- testes de integração com **Testcontainers reais de PostgreSQL e Kafka** (e Redis, que o PulsePass usa);
- ambiente completo orquestrado via **`docker-compose.yml`** (backend, frontend, bancos, Kafka e UI de mensageria);
- testes de carga como bônus desejável.

O grupo acrescentou no Pacote 1 metas próprias (rotuladas como **meta do grupo** abaixo): ArchUnit (RNF34), PIT ≥ 70% (RNF32), ≥ 90% de ramos na FSM (RNF04), gitleaks (RNF23), varredura de dependências (RNF24), Lighthouse (RNF43) e publicação de imagens no GHCR (RNF40).

4.2 Fluxo de ponta a ponta

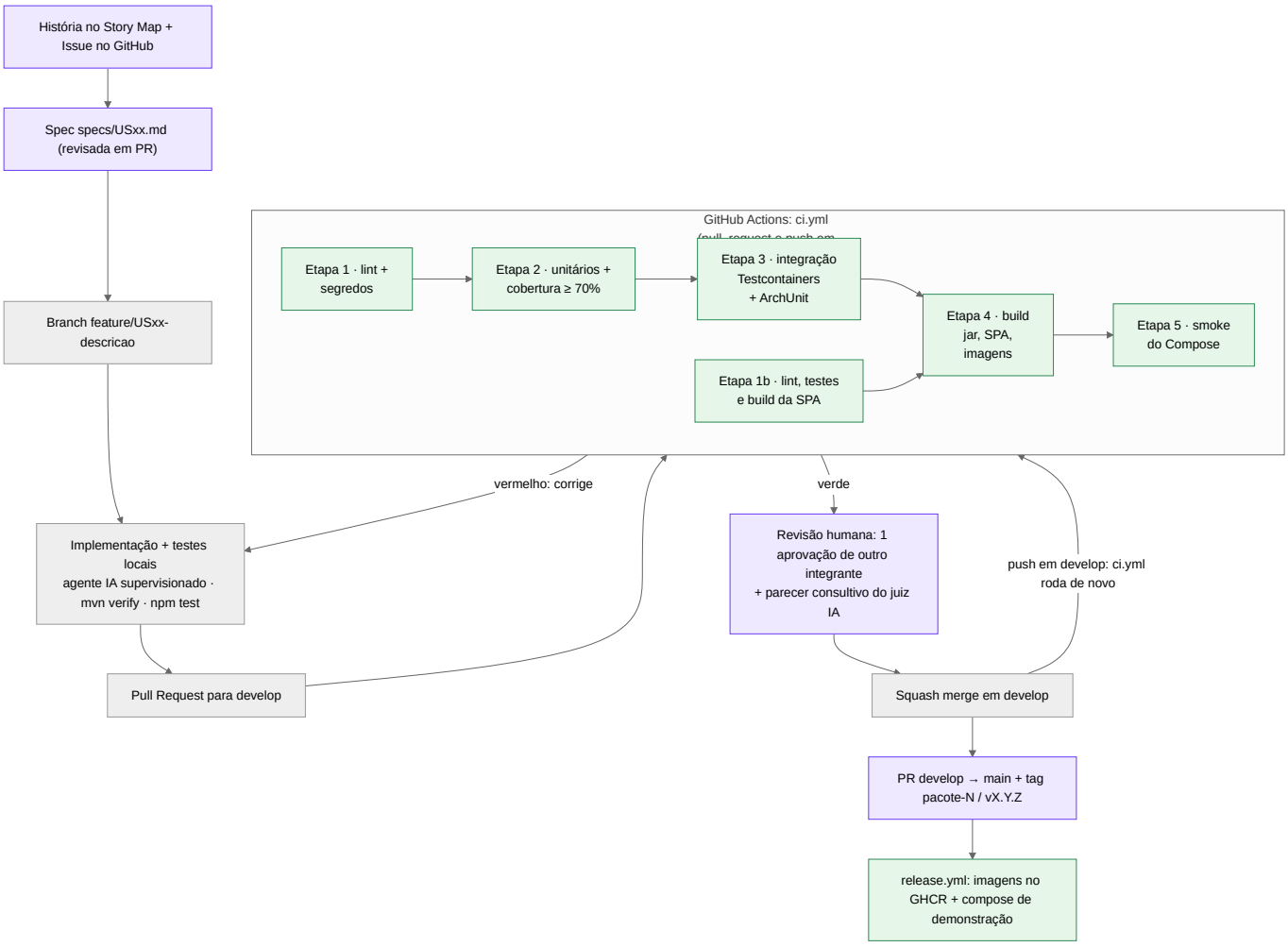


Figura 22. Pipeline de desenvolvimento. Roxo: portões humanos. Verde: etapas automatizadas.

O merge de um PR em `develop` não é um evento próprio do GitHub Actions: ele gera um evento `push` em `develop`. Por isso o workflow usa os gatilhos `pull_request` (destinos `develop` e `main`) e `push` (branch `develop`) **[Proposta para revisão do grupo]** (ADR-006).

4.3 Jobs da CI

#	Job	Depende de	Ambiente	Comandos previstos	Falha quando	Artefatos	Origem
1	`lint`	-	`ubuntu-latest`, Temurin 21, cache Maven	`mvn -B -ntp spotless:check checkstyle:check`; `gitleaks detect --redact`	Qualquer violação de estilo ou segredo encontrado	Relatório Checkstyle	[Enunciado] lint + meta do grupo (RNF23, RNF35)
1b	`frontend`	-	Node 22, cache npm	`npm ci`; `npm run lint`; `npm run typecheck`; `npm test -- --coverage`; `npm run build`	Erro de ESLint/Prettier, tipo, teste ou build	`spa-dist`, cobertura	[Enunciado] lint, testes e compilação
2	`unit`	`lint`	Temurin 21	`mvn -B -ntp test jacoco:report jacoco:check`	Teste falhou; cobertura de linhas do pacote `domain` < 70%; ramos da FSM < 90%	`jacoco.xml` por serviço, Surefire	[Enunciado] ≥ 70% de domínio + meta do grupo (RNF04, RNF30)
3	`integration`	`unit`	`ubuntu-latest` (Docker já disponível no runner)	`mvn -B -ntp verify -Pintegration` (Testcontainers: `postgres:16`, `apache/kafka:3.8.0`, `redis:7`) + regras ArchUnit	Teste de integração ou regra de arquitetura falhou	Failsafe, logs dos contêineres em falha	[Enunciado] Testcontainers + meta do grupo (RNF31, RNF34)
4	`build`	`integration`, `frontend`	Temurin 21, Docker Buildx	`mvn -B -ntp -DskipTests package`; `docker buildx build` de cada serviço (sem publicar)	Compilação ou build de imagem falhou	JARs, imagens em cache do Buildx	[Enunciado] compilação
5	`compose-smoke`	`build`	Runner com Docker	`cp .env.example .env`; `docker compose up -d --wait --wait-timeout 300`; script de *smoke* (health de todos, login de teste, GET `/eventos`); `docker compose down -v`	Algum serviço não fica saudável em 5 min ou o *smoke* falha	`docker compose logs` em falha	Meta do grupo (RNF17, RNF37)
6	`security`	`build`	-	`trivy image` nas imagens; OWASP Dependency-Check	Vulnerabilidade crítica (só bloqueia PR para `main`)	Relatório SARIF	Meta do grupo (RNF24)
7	`mutation`	`unit`	-	`mvn -B -ntp -pl inventory-service,checkout-service org.pitest:pitest-maven:mutationCoverage`	*Mutation score* < 70% (só em PR para `main` e agendado semanal)	Relatório PIT	Meta do grupo (RNF32)
8	`ai-review`	-	Ação de revisão por IA	Comenta o PR comparando o diff com a spec	Nunca bloqueia (consultivo)	Comentário no PR	Meta do grupo (Pacote 1, 1.6)

Duração-alvo: ≤ 15 min (RNF39). Testes de carga (k6) **não** fazem parte da CI de PR: ficam em workflow manual (`workflow_dispatch`) e são bônus do enunciado (RNF33).

4.3.1 Detalhes de execução

Tema	Decisão **[Proposta para revisão do grupo]**
Docker para Testcontainers	Runners `ubuntu-latest` do GitHub já têm Docker; o Testcontainers usa o *socket* padrão. Ryuk fica ligado para limpar contêineres
Versões fixadas	Imagens com tag exata (`postgres:16.4`, `apache/kafka:3.8.0`, `redis:7.4`, `quay.io/keycloak/keycloak:26.0`); ações do GitHub fixadas por versão maior e revisadas pelo Dependabot
Caches	`actions/setup-java` com cache Maven; `actions/setup-node` com cache npm; cache de camadas do Buildx (`type=gha`)
Monorepo	Maven multi-módulo na raiz (`services/*`) + `frontend/`; `paths` no workflow evita rodar o job da SPA quando só o backend muda (o job obrigatório é marcado como sucesso por um *job* de agregação)
Relatórios	Cobertura e testes publicados como artefatos e como resumo do job (`$GITHUB_STEP_SUMMARY`)
Segredos	O CI não precisa de segredos reais: Testcontainers e Compose usam credenciais geradas para o teste. `GITHUB_TOKEN` só no `release.yml` para o GHCR
Permissões	`permissions: contents: read` por padrão; `packages: write` só no `release.yml`

4.3.2 Esboço do workflow (previsto, ainda não versionado)

O arquivo abaixo **não** foi criado em `.github/workflows/` porque os projetos Maven e npm que ele chama ainda não existem; criar o workflow agora geraria execuções vermelhas sem significado. Ele será adicionado junto com o esqueleto dos serviços (US33).

```yaml
name: ci
on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop]
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: mvn -B -ntp spotless:check checkstyle:check
      - uses: gitleaks/gitleaks-action@v2
  frontend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: frontend } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22', cache: npm, cache-dependency-path: frontend/package-lock.json }
      - run: npm ci && npm run lint && npm run typecheck && npm test -- --coverage && npm run build
  unit:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: mvn -B -ntp test jacoco:report jacoco:check
  integration:
    needs: unit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: mvn -B -ntp verify -Pintegration
  build:
    needs: [integration, frontend]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: mvn -B -ntp -DskipTests package
      - uses: docker/setup-buildx-action@v3
      - run: docker buildx bake --file docker-bake.hcl
  compose-smoke:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cp .env.example .env
      - run: docker compose up -d --build --wait --wait-timeout 300
      - run: ./scripts/smoke.sh
      - if: failure()
        run: docker compose logs --no-color > compose.log
      - run: docker compose down -v
```

4.4 Ambiente Docker Compose

O `docker-compose.yml` é o ambiente de desenvolvimento e de demonstração **[Enunciado]**, com os serviços listados no RNF37 **[Decisão do grupo, Pacote 1]**. A tabela abaixo é o desenho proposto.

Serviço Compose	Imagem / build	Rede	Volume	Saúde (healthcheck)	Depende de (saudável)	Porta no host
`postgres`	`postgres:16.4`	`data`	`pg-data`	`pg_isready`	-	-
`postgres-init`	script SQL montado em `/docker-entrypoint-initdb.d`	`data`	-	(roda na 1ª subida do Postgres)	-	-
`redis`	`redis:7.4` (`--appendonly yes`, ACL via arquivo)	`data`	`redis-data`	`redis-cli ping`	-	-

Serviço Compose	Imagem / build	Rede	Volume	Saúde (healthcheck)	Depende de (saudável)	Porta no host
`kafka`	`apache/kafka:3.8.0` (KRaft, 1 broker)	`data`	`kafka-data`	`kafka-broker-api-versions.sh`	-	-
`kafka-init`	mesma imagem; cria tópicos e DLQs	`data`	-	termina com código 0	`kafka`	-
`kafka-ui`	`kafbat/kafka-ui`	`data`, `ops`	-	HTTP `/actuator/health`	`kafka`	8089
`keycloak`	`quay.io/keycloak/keycloak:26.0` com `--import-realm`	`backend`, `data`	realm em `infra/keycloak/pulsepass-realm.json`	HTTP `/health/ready`	`postgres`	8180
`mailpit`	`axllent/mailpit`	`backend`	-	HTTP `/livez`	-	8025 (UI)
`payment-simulator`	build `services/payment-simulator`	`backend`	-	`/actuator/health`	-	-
`iam-service`, `inventory-service`, `checkout-service`, `ticket-service`, `telemetry-service`	build `services/<nome>`	`backend`, `data`	`ticket-pdfs` (só Ticket)	`/actuator/health/readiness`	`postgres`, `redis`, `kafka-init` (concluído), `keycloak`	-
`realtime-service`	build `services/realtime-service`	`backend`, `data`	-	`/actuator/health/readiness`	`kafka`, `keycloak`	-
`api-gateway`	build `services/api-gateway`	`edge`, `backend`	-	`/actuator/health`	serviços de domínio	-
`frontend`	build `frontend/` (Nginx servindo a SPA, TLS local e proxy para `/api` e `/ws`)	`edge`	-	HTTP `/`	`api-gateway`, `realtime-service`	8443
`otel-collector`, `prometheus`, `grafana`, `jaeger`	imagens oficiais com configuração em `infra/`	`ops`, `backend`	`prom-data`, `grafana-data`	HTTP de cada um	-	3000 (Grafana), 16686 (Jaeger)

Tema	Desenho
Redes	`edge` (só Nginx e Gateway), `backend` (serviços), `data` (bancos e broker), `ops` (observabilidade). Serviços de domínio não publicam portas no host
Inicialização	`postgres-init` cria os bancos `iam`, `inv`, `chk`, `tkt`, `tel`, `keycloak` e um usuário por banco; cada serviço aplica suas migrações Flyway ao subir; `kafka-init` cria tópicos com partições e DLQs; Keycloak importa o realm com papéis, clientes e usuários de teste
Configuração	Variáveis em `.env` (fora do Git); `.env.example` com valores de exemplo sem segredos reais (RNF23, RNF38). Perfis Spring `local` e `ci`
Recursos	Limites de memória por serviço (`mem_limit`: 512 MB por serviço Java, 1 GB Kafka, 768 MB Keycloak). Estimativa total ≈ 8 a 9 GB, compatível com o notebook de referência de 16 GB do Pacote 1
Escala	`docker compose up --scale inventory-service=2 --scale ticket-service=2` sem mudar código (RNF13); o Gateway resolve réplicas pelo DNS do Compose
Comando de subida (previsto)	`cp .env.example .env && docker compose up -d --wait`

4.5 Build de imagem, publicação e implantação

Etapa	Quando	Onde	Estado
Build de imagem	Todo PR e push em `develop` (job `build`)	Runner do GitHub, sem publicar	Planejado
Publicação	Tag `pacote-N` ou `vX.Y.Z` em `main` (`release.yml`)	GHCR, tags `sha-<commit>` e versão (RNF40)	Planejado
Implantação	Manual: `docker compose -f docker-compose.yml -f docker-compose.demo.yml up -d` usando as imagens publicadas	Máquina de um integrante ou laboratório	Planejado

Sem implantação em nuvem [Decisão do grupo, Pacote 1 (P07)]: a Visão define execução local/acadêmica. O fórum da disciplina sobre e-mails para a AWS Academy (26/10/2026) não muda sozinho o escopo desta entrega; se o professor pedir implantação na AWS, o grupo registrará um novo ADR.

4.6 Compromisso da iteração 2: estado real

O processo do Pacote 1 (seção 1.1) previu para a iteração 2 o "esqueleto dos serviços + docker-compose + CI". Esse compromisso **não faz parte da lista do professor para o Pacote 2** (que pede descrições), mas registramos o estado com honestidade:

Item	Estado em 04/11/2026	Próximo passo	História
Esqueleto Maven multi-módulo dos serviços	Não iniciado	Gerar módulos com Spring Initializr e camadas hexagonais vazias	US32
`docker-compose.yml`	Não iniciado	Subir infraestrutura (Postgres, Redis, Kafka, Kafka UI, Keycloak, Mailpit) primeiro, depois os serviços	US32
`.github/workflows/ci.yml`	Não iniciado (esboço na seção 4.3.2)	Versionar junto com o esqueleto, para não ter CI apontando para arquivos inexistentes	US33
Branch `develop` e proteção de branch	Não existe; proteção não verificada	Integrante com permissão de admin cria e configura (documento 5)	US33

Esta entrega priorizou os cinco documentos pedidos pelo professor. **[Proposta para revisão do grupo]**: o esqueleto passa a ser a primeira tarefa da iteração 3, antes das histórias da fatia MVP.

4.7 Rastreabilidade

Requisito	Seção
CI a cada PR ou merge em `develop` com lint, testes e compilação [Enunciado], RNF39, US33	4.2, 4.3
≥ 70% de cobertura de domínio [Enunciado], RNF30	4.3 (job `unit`)
Testcontainers PostgreSQL e Kafka (e Redis) [Enunciado], RNF31	4.3 (job `integration`)
Compose completo [Enunciado], RNF17, RNF37, US32	4.4
Testes de carga (bônus) [Enunciado], RNF33	4.3 (workflow manual)
RNF23, RNF24, RNF32, RNF34, RNF35, RNF40	4.3, 4.5

5. Descrição do Processo de GCS (Gerenciamento da Configuração de Software)

> Versão 1.0 · 04/11/2026 · Grupo 9 · Será atualizado nos Pacotes 3 e 4. Rótulos: **[Enunciado]** · **[Decisão do grupo, Pacote 1]** · **[Proposta para revisão do grupo]**. O enunciado pede: ferramenta, descrição do fluxo de trabalho (branches, regras de uso, ...). O fluxo de branches e as regras de PR foram definidos no Pacote 1 (seção 1.7) e são mantidos aqui; este documento detalha o processo e separa **regra definida** de **configuração técnica já ativa**.

5.1 Ferramentas

Ferramenta	Papel no GCS
Git	Controle de versão de tudo que é item de configuração
GitHub (repositório `Mateus-Tieppo/construcao-de-software`)	Repositório remoto, Pull Requests, revisão, proteção de branches
GitHub Issues + Projects	Pedidos de mudança, backlog da iteração, ligação com histórias (US) e RNFs
GitHub Actions	Verificação automática de cada mudança (documento 4)
GHCR	Registro das imagens versionadas (RNF40)
Conventional Commits	Padrão de mensagens que permite gerar notas de versão
Dependabot	Pedidos de atualização de dependências e ações
Claude Code	Agente de IA que escreve código, specs e documentos sob supervisão; nunca faz merge

5.2 Itens de configuração

Item	Onde fica	Controle
Specs das histórias	`specs/USxx-*.md`	PR com revisão; mudança de comportamento sem spec atualizada reprova o PR (Pacote 1, 1.7)
Código-fonte dos serviços e da SPA	`services/*`, `frontend/`	PR + CI + revisão
Contratos	OpenAPI gerado no build; eventos em `contratos/` e JSON Schemas	Mudança incompatível exige nova versão (v2), ver documento 3
Migrações de banco	`services/<s>/src/main/resources/db/migration` (Flyway, `V<n>__descricao.sql`)	Nunca editar migração já mesclada em `develop`; correção é migração nova
Infraestrutura	`docker-compose.yml`, `infra/` (realm Keycloak, Prometheus, Grafana, tópicos)	PR + job `compose-smoke`
Pipelines	`.github/workflows/*`	PR com revisão de quem conhece a pipeline
Dependências	`pom.xml`, `package-lock.json`, `pacote-2/package-lock.json`	Versões fixadas; atualização por PR
Documentação e ADRs	`pacote-N/`, `pacote-N/adr/`, `CLAUDE.md`	PR com revisão
Entregas	`pacote-N/PacoteN_Grupo9.pdf` + tag `pacote-N`	Baseline
Enunciado	`contexto/`	Cópia do Moodle: não é alterada pelo grupo
Fora do Git	`.env`, chaves, senhas, tokens	`.gitignore` + gitleaks no CI (RNF23)

5.3 Modelo de branches

[Decisão do grupo, Pacote 1 (1.7)]: Git Flow simplificado.

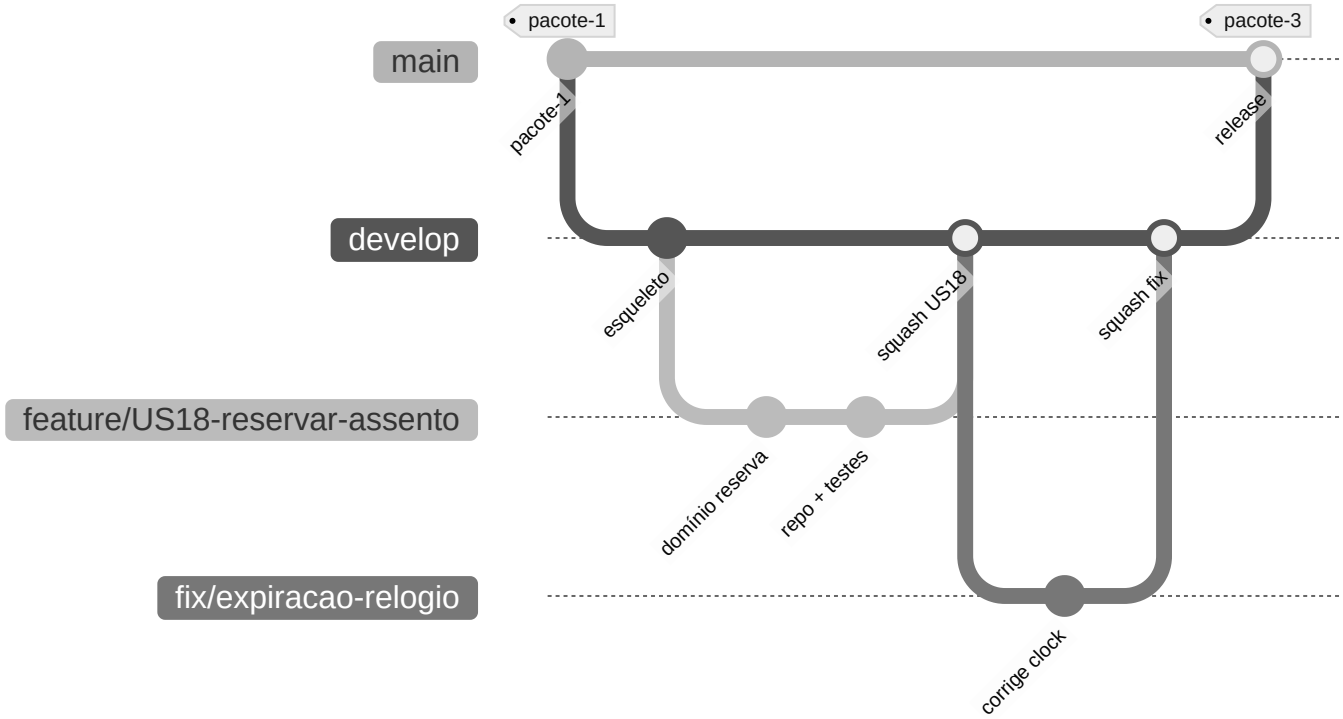


Figura 23. Branches e baselines. Exemplo ilustrativo; as branches `develop` e `feature/*` ainda não existem.

Branch	Para que serve	Nasce de	Volta para	Regra
`main`	Versões entregues (uma por pacote)	-	-	Só recebe PR vindo de `develop` (ou `docs/*` enquanto não houver código, ver 5.8); cada merge recebe tag
`develop`	Integração contínua do trabalho	`main`	`main`	Só recebe PR com CI verde e 1 aprovação
`feature/USxx-descricao`	Uma história do backlog	`develop`	`develop`	Uma história por branch; vida curta (≤ 1 semana)
`fix/descricao`	Correção de defeito	`develop` (ou `main`, se for na entrega)	origem	Precisa de teste que reproduz o defeito
`docs/descricao`	Só documentação	`develop` ou `main`	origem	Mesmo fluxo de PR

Proibido: *push* direto em `main` e `develop`, *force push* nessas branches, merge sem PR.

5.4 Ciclo de uma mudança

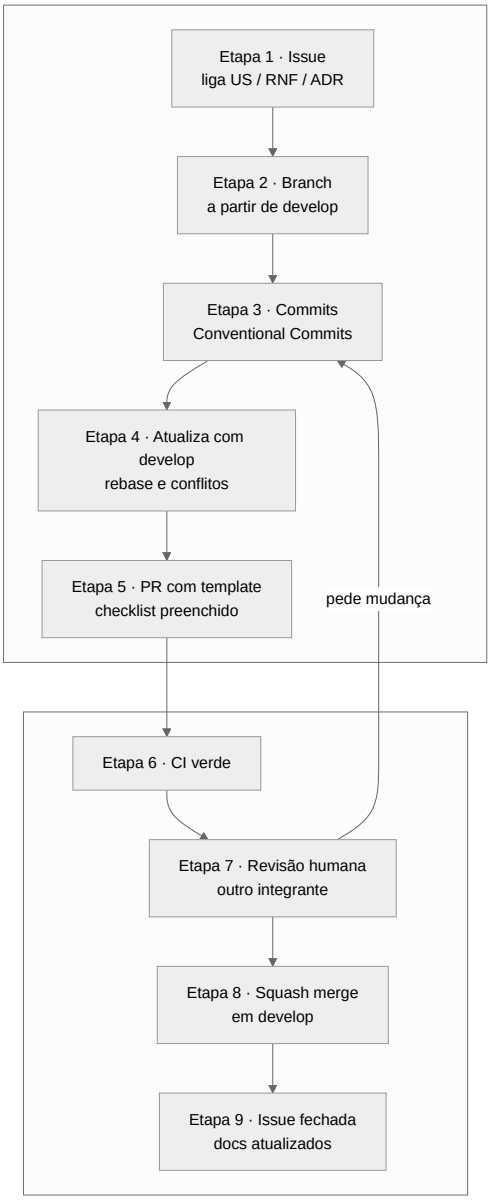


Figura 24. Ciclo de uma mudança.

Passo	Regra
1. Issue	Toda mudança nasce de uma issue com o ID da história (`US18`) ou do RNF; bug traz passos para reproduzir
2. Branch	Nome `feature/US18-reservar-assento`; criada a partir de `develop` atualizado
3. Commits	`tipo(escopo): resumo` em português, ex.: `feat(inventario): reserva atômica de assentos (US18)`. Tipos: `feat`, `fix`, `docs`, `test`, `refactor`, `chore`, `ci`. Commits gerados com IA levam o trailer `Co-Authored-By` do agente
4. Atualização	`git rebase develop` antes de abrir o PR; conflitos resolvidos por quem é dono da branch, com os testes rodando de novo
5. PR	Título igual ao commit final; descrição com `Closes #<issue>`; template da seção 5.5 preenchido
6. CI	Todos os jobs obrigatórios verdes (documento 4)
7. Revisão	Pelo menos 1 aprovação de **outro** integrante humano. O parecer do juiz de IA é consultivo. Quem escreveu a spec ou o código não aprova o próprio PR
8. Merge	*Squash merge* em `develop` (histórico linear, um commit por história); branch apagada após o merge
9. Fechamento	Issue fechada pelo PR; se mudou comportamento, spec e documentação do pacote atualizadas no mesmo PR

5.5 Template de Pull Request

O arquivo `.github/pull_request_template.md` (a criar com o esqueleto, US33) terá:

```
## O que muda
Closes #<issue> · História: USxx · Spec: specs/USxx-*.md · ADRs: ...

## Checklist
- [ ] Escopo limitado à história; arquivos fora do escopo explicados
- [ ] Contratos: OpenAPI e eventos atualizados; mudança incompatível virou v2
- [ ] Migração nova (sem editar migração antiga), se mexeu em banco
- [ ] Testes: unitários do domínio, integração com Testcontainers quando toca banco, Kafka ou Redis
- [ ] Evidência: print, log ou saída do teste dos critérios de aceitação da spec
- [ ] Sem segredos, tokens ou dados pessoais reais no diff
- [ ] Documentação e spec atualizadas
- [ ] Uso de IA: o que foi gerado por agente e o que o autor verificou manualmente
```

A linha de uso de IA cumpre o processo do Pacote 1: o agente gera, o humano verifica e responde pelo código no review.

5.6 Versionamento, baselines e releases

Conceito	Regra **[Proposta para revisão do grupo, sobre o Pacote 1]**
Baseline de entrega	Tag anotada `pacote-1` … `pacote-4` no commit de `main` que gerou o PDF entregue
Versão do produto	SemVer v`MAJOR.MINOR.PATCH` a partir do Pacote 3 (`v0.3.0` no Pacote 3, `v1.0.0` no Pacote 4)
Notas de versão	GitHub Release gerado a partir dos Conventional Commits desde a tag anterior
Imagens	Publicadas com `sha-<commit>` e com a versão, a partir da tag (documento 4)
Reprodução de uma entrega	`git checkout pacote-N` + `docker compose up` com as imagens daquela tag; o PDF da entrega é regenerado com `npm ci && npm run pdf` dentro de `pacote-N/`
Eventos e APIs	Versão no nome do tópico (`.v1`) e no caminho da API quando houver quebra (`/v2/...`)

5.7 Configuração, segredos e mudanças

Tema	Regra
Segredos	Nunca no Git. `.env` local; `.env.example` só com nomes e valores de exemplo; segredos de CI (quando houver) em *GitHub Secrets*
Configuração por ambiente	Perfis Spring `local`, `ci`, `demo`; variáveis de ambiente sobrescrevem tudo (RNF38)
Migrações	Flyway versionado; a mesma sequência roda em dev, CI e demonstração
Mudança de contrato	Discutida na issue; consumidores atualizados antes de o produtor remover algo
Rollback	`git revert` do commit de *squash* em `develop` (nunca reescrever histórico publicado); para uma entrega já marcada, nova tag de correção
Débito e exceções	Desvio consciente de uma regra vira issue com rótulo `debito-tecnico`

5.8 Estado real da configuração (04/11/2026)

Regra	Definida	Configurada no GitHub	Observação
Branches `main` e `develop`	Sim (Pacote 1)	Só `main` existe	`develop` será criada com o esqueleto (US33)
PR obrigatório, 1 aprovação, CI verde	Sim (Pacote 1)	**Não verificado**	Requer integrante com permissão de admin; não foi alterado nesta entrega
Workflows de CI	Sim (documento 4)	Não existem	Esboço na seção 4.3.2
Tags de baseline	Sim	**Não**: o repositório remoto não tem nenhuma tag (`git ls-remote --tags`, 04/11/2026)	Criar `pacote-1` (commit `644f965`) e `pacote-2` em `main` após a aprovação do grupo
Fluxo de PR para documentos	Sim	Em uso	O plano do Pacote 2 entrou por PR (`docs/plano-pacote-2` → `main`, PR #1); este pacote entra por `docs/pacote-2` → `main`

Enquanto não houver código nem `develop`, as entregas documentais seguem `docs/*` → PR → `main`, com revisão de outro integrante antes do merge **[Proposta para revisão do grupo]** (ADR-006). As regras de proteção estão descritas como processo; nenhuma aprovação ou proteção técnica é alegada sem ter sido configurada.

5.9 Rastreabilidade

Exigência	Seção
Ferramenta [Enunciado]	5.1
Fluxo de trabalho: branches [Enunciado]	5.3
Regras de uso [Enunciado]	5.3, 5.4, 5.5, 5.7
Revisão humana e uso de IA (Pacote 1, 1.4 e 1.7)	5.4, 5.5
Baselines e reprodução de entregas	5.6
RNF23 (sem segredos), RNF38 (configuração externalizada), RNF39 (proteção de branch)	5.7, 5.8